Apollo Website Automation Framework

Using Selenium, Pytest & Behave BDD

Capstone Project Report

Submitted By	RAUSHAN KUMAR
Superset ID	5531902
Project Title	Apollo Website Automation

Apollo 247 Automation Testing Documentation

Selenium PyTest Project + Behave BDD Project

Combined Capstone Project Test Documentation

Document Scope

This document explains both Apollo automation projects included in the CapstoneProject package: the Selenium PyTest framework and the Behave BDD framework. It includes framework structure, test flow, positive and negative cases, E2E scenarios, Allure reporting evidence, data-driven testing, logs/screenshots, challenges, and future scope.

Project	Framework	Primary Test Style	Report
ApolloAutomation	Selenium WebDriver + PyTest + POM	test_e2e.py and test_positive_negative.py	Allure PyTest
BBD_Apollo	Selenium WebDriver + Behave BDD + POM	Gherkin feature files and step definitions	Allure Behave

Table of Contents

1. Project Overview

The Apollo 247 / Apollo Pharmacy automation project validates the shopping workflow around category navigation, Health Monitors, Doctor S Choice filtering, product addition to cart, cart validation, and proceed flow. The same business workflow is implemented in two formats: a Selenium PyTest framework and a Behave BDD framework.

2. Objectives

- Automate Apollo 247 login flow with mobile number and manual OTP validation.
- Validate positive and negative category visibility scenarios.
- Automate Health Monitors navigation and Doctor S Choice brand filtering.
- Add a product to cart and verify that the same product is present in cart.
- Proceed from cart to the next checkout/cart step.
- Generate Allure reports and maintain supporting report/project PDFs as test evidence.
- Show two framework styles: PyTest and BDD Behave.

3. Technologies Used

Technology	Purpose
Python	Core programming language for automation logic
Selenium WebDriver	Browser automation and UI interaction
PyTest	Test runner for ApolloAutomation project
Behave	BDD test runner for BBD_Apollo project
Allure PyTest / Allure Behave	Interactive test reporting
Page Object Model	Reusable page classes and separation of test logic
CSV	External test data for login and category cases
LogGen / logging	Execution logs and debugging information
webdriver-manager	Automatic ChromeDriver management

4. Test Environment

Parameter	Value
Operating System	Windows 10 / 11
Browser	Google Chrome
IDE	PyCharm
Application URL	https://www.apollo247.com/
Report Tool	Allure
Execution Mode	Non-headless Chrome by default

5. Project 1 - ApolloAutomation: Selenium PyTest Framework

ApolloAutomation is the PyTest-based Selenium framework. It uses Page Object Model, reusable utilities, CSV test data, ordered test execution, screenshots, logs, and Allure reporting.

5.1 Selenium Project Folder Structure

```
ApolloAutomation/
├── config/
│   └── config.properties
├── data/
│   ├── login_data.csv
│   └── shop_by_category_data.csv
├── logs/
│   └── automation.log
├── pages/
│   ├── basepage.py
│   ├── loginpage.py
│   └── shopbycategorypage.py
├── reports/
│   ├── allure-results/
│   ├── allure-report/
│   └── screenshots/
├── tests/
│   ├── test_e2e.py
│   └── test_positive_negative.py
├── utils/
│   ├── config_reader.py
│   ├── csv_reader.py
│   ├── logger.py
│   └── screenshot_util.py
├── conftest.py
├── pytest.ini
└── requirements.txt
```

5.2 Selenium Key Files and Usage

File / Folder	Purpose
config/config.properties	Stores browser, base URL, timeout, implicit wait, and headless settings.
data/login_data.csv	Stores mobile number and expected login result.
data/shop_by_category_data.csv	Stores positive and negative category visibility data.
pages/basepage.py	Common Selenium methods such as click, wait, type, scroll, JS click, popup removal.
pages/loginpage.py	Login workflow: open login popup, enter mobile, verify OTP screen, submit OTP.
pages/shopbycategorypage.py	Health Monitors, brand filter, add-to-cart, cart validation, proceed flow.
tests/test_e2e.py	End-to-end PyTest test cases: login, category, filter, cart, proceed.
tests/test_positive_negative.py	Parameterized positive and negative category test cases.
utils/screenshot_util.py	Captures screenshots for passed/failed tests.
conftest.py	PyTest fixture setup/teardown, browser launch, screenshot attachment to Allure.
pytest.ini	PyTest configuration and Allure result directory.

5.3 Selenium Page Classes and Methods

Class / Utility	Important Methods
BasePage	get_element(), click(), type(), is_visible(), scroll_to_element(), js_click(), close_popup_if_present()

LoginPage	open_login_popup(), login_with_mobile_number(), is_otp_screen_visible(), wait_for_otp_entry_and_submit(), submit_otp()
ShopByCategoryPage	is_category_visible(), click_health_monitors(), apply_doctor_s_choice_filter(), add_exact_product_to_cart_with_retry(), go_to_cart(), click_proceed_button()
ConfigReader	get(), get_base_url(), get_browser(), get_timeout(), is_headless()
CSVReader	read_csv()
ScreenshotUtil	capture_screenshot()

5.4 Selenium Test Files

Test File	Type	What It Validates
tests/test_e2e.py	End-to-End	Login, Health Monitors navigation, Doctor S Choice filter, product add to cart, cart verification, proceed flow.
tests/test_positive_negative.py	Positive + Negative	Valid categories are visible; invalid categories/products are not visible.

5.5 Selenium Data-Driven Testing

CSV File	Sample Data / Usage
login_data.csv	mobile_number = 8757726775, expected_result = success
shop_by_category_data.csv	Health Monitors/Pain Relief/Baby Care = visible; Fake Category/Invalid Product = not_visible

6. Project 2 - BBD_Apollo: Behave BDD Framework

BBD_Apollo implements the same Apollo workflow using Behaviour Driven Development. Test scenarios are written in Gherkin language inside feature files, and Python step definition files connect those English steps to Selenium page-object methods.

6.1 BDD Project Folder Structure

```
BBD_Apollo/
├── config/
│   └── config.ini
├── data/
│   ├── login_data.csv
│   └── shop_by_category_data.csv
├── features/
│   ├── login.feature
│   ├── shop_by_category.feature
│   ├── environment.py
│   └── steps/
│       ├── login_steps.py
│       └── shop_steps.py
├── locators/
│   ├── login_locators.py
│   └── shop_locators.py
├── logs/
│   └── automation.log
├── pages/
│   ├── base_page.py
│   ├── login_page.py
│   └── shop_by_category_page.py
├── reports/
│   ├── allure-results/
│   ├── allure-report/
│   └── screenshots/
├── utils/
│   ├── config_reader.py
│   ├── csv_reader.py
│   ├── logger.py
│   ├── screenshot_util.py
│   └── waits.py
├── behave.ini
├── requirements.txt
└── runtests.py
```

6.2 BDD Key Files and Usage

File / Folder	Purpose
config/config.ini	Stores browser, base URL, wait, timeout, headless flag, and mobile number under [app].
features/login.feature	BDD login scenario written using Given/When/Then.
features/shop_by_category.feature	Category validation, Health Monitors, filter, cart, proceed scenarios.
features/steps/login_steps.py	Python implementation for login.feature steps.
features/steps/shop_steps.py	Python implementation for shop_by_category.feature steps.
features/environment.py	Behave setup/teardown: browser start, scenario screenshots, Allure attachments, browser close.
locators/login_locators.py	Login page locators.
locators/shop_locators.py	Shop/category/cart locators.
pages/base_page.py	Common Selenium actions used by all page classes.
pages/login_page.py	Login page methods.

pages/shop_by_category_page.py	Shop/category/cart page methods.
runtests.py	Runs Behave with Allure formatter and generates Allure HTML report.

6.3 BDD Feature Files

Feature File	Scenarios
features/login.feature	Login with valid mobile number and OTP.
features/shop_by_category.feature	Validate category visibility; open Health Monitors; apply Doctor S Choice filter; add product to cart; proceed after adding product.

6.4 Steps Files - Purpose

In Behave, feature files contain English test steps, but these steps must be mapped to Python functions. The steps folder acts as a bridge between Gherkin steps and Selenium automation code.

Steps File	Responsibilities
features/steps/login_steps.py	Opens application, enters mobile number, validates OTP screen, waits for manual OTP entry, submits login.
features/steps/shop_steps.py	Validates category visibility, opens Health Monitors, applies filter, adds product to cart, verifies cart, clicks Proceed.

6.5 BDD Page Classes and Methods

Class / File	Important Methods
BasePage	get_element(), click(), type_text(), is_visible(), scroll_to_element(), js_click(), close_popup_if_present()
LoginPage	open_login_popup(), login_with_mobile_number(), is_otp_screen_visible(), wait_for_otp_entry_and_submit()
ShopByCategoryPage	is_category_visible(), click_health_monitors(), apply_doctor_s_choice_filter(), add_exact_product_to_cart_with_retry(), go_to_cart(), click_proceed_button()
environment.py	before_all(), before_scenario(), after_scenario(), after_all()

7. Positive and Negative Test Cases

7.1 Positive Test Cases

Case	Project Location	Expected Result
Health Monitors category visible	Selenium: test_positive_negative.py / BDD: shop_by_category.feature	Category should be visible.
Pain Relief category visible	Selenium: test_positive_negative.py / BDD: shop_by_category.feature	Category should be visible.
Baby Care category visible	Selenium: test_positive_negative.py / BDD: shop_by_category.feature	Category should be visible.
Login with valid mobile number	Selenium: test_e2e.py / BDD: login.feature	OTP screen should appear and OTP submission should succeed.
Add Doctor S Choice product to cart	Selenium: test_e2e.py / BDD: shop_by_category.feature	Selected product should be present in cart.

7.2 Negative Test Cases

Case	Input Data	Expected Result
Fake Category	category_name = Fake Category, expected_result = not_visible	The category should not be visible; assertion expects False.
Invalid Product	category_name = Invalid Product, expected_result = not_visible	The product/category should not be visible; assertion expects False.

Explanation: Negative testing verifies that invalid or non-existing items are not treated as valid. If Fake Category or Invalid Product appears on the page, the test should fail because the expected result is not_visible.

8. End-to-End Test Flow

Step	Selenium PyTest Flow	BDD Behave Flow
1	test_login[data0] reads login_data.csv and enters mobile number.	login.feature: Given user opens Apollo application / When user enters valid mobile number.
2	OTP screen is verified and OTP is submitted manually.	Then OTP screen should be visible / When user enters OTP manually.
3	Health Monitors category is opened.	Scenario: Open Health Monitors category.
4	Brands filter is verified and Doctor S Choice filter is applied.	Scenario: Apply Doctor S Choice filter.
5	Product is added to cart using add_exact_product_to_cart_with_retry().	Scenario: Add Doctor S Choice product to cart.
6	Cart is opened and selected product is verified.	Then cart page should be opened / selected product should be present in cart.
7	Proceed button is clicked and next cart step is validated.	Scenario: Proceed after adding product to cart.

9. Allure Report Execution Summary

9.1 Selenium PyTest - ApolloAutomation

Metric	Value
Total test cases/scenarios	10
Passed	10
Failed	0
Success Rate	100%
Total measured test duration	204.1 seconds

Test / Scenario Name	Status	Duration

test_shop_by_category_data[data2]	PASSED	1.5s
test_shop_by_category_data[data1]	PASSED	1.7s
test_shop_by_category_data[data0]	PASSED	1.8s
test_shop_by_category_data[data4]	PASSED	3.8s
test_shop_by_category_data[data3]	PASSED	4.2s
test_click_health_monitors_category	PASSED	18.5s
test_login[data0]	PASSED	21.3s
test_apply_doctor_s_choice_filter_after_health_monitors	PASSED	38.7s
test_add_doctor_s_choice_product_to_cart	PASSED	49.7s
test_proceed_after_adding_product_to_cart	PASSED	62.7s

9.2 Behave BDD - BBD_Apollo

Metric	Value
Total test cases/scenarios	10
Passed	10
Failed	0
Success Rate	100%
Total measured test duration	197.3 seconds

Test / Scenario Name	Status	Duration
Validate category visibility -- @1.3	PASSED	1.6s
Validate category visibility -- @1.2	PASSED	2.1s
Validate category visibility -- @1.1	PASSED	2.6s
Validate category visibility -- @1.5	PASSED	3.8s
Validate category visibility -- @1.4	PASSED	3.8s
Open Health Monitors category	PASSED	18.2s
Login with valid mobile number and OTP	PASSED	26.6s
Apply Doctor S Choice filter	PASSED	34.5s
Add Doctor S Choice product to cart	PASSED	46.0s
Proceed after adding product to cart	PASSED	58.1s

10. Allure Reporting and Evidence

- Selenium project uses allure-pytest and PyTest hooks in conftest.py to attach screenshots to Allure.
- BDD project uses allure-behave formatter and environment.py to attach screenshots/logs per scenario.
- Allure results are stored in reports/allure-results.
- Final generated HTML reports are stored in reports/allure-report.
- Screenshots are stored in reports/screenshots and attached as evidence.

11. Commands Used

Project	Command	Purpose
ApolloAutomation	pytest -v -s	Run PyTest test suite; pytest.ini writes Allure results.
ApolloAutomation	allure serve reports/allure-results	Open Allure report from generated results.
BBD_Apollo	python runtests.py	Run Behave scenarios and generate Allure HTML report automatically.
BBD_Apollo	allure open reports/allure-report	Open already-generated BDD Allure report.

12. Selenium / Automation Concepts Used

Concept	Usage
Explicit Wait	Waits for elements to be visible/clickable before interaction.
XPath	Used for dynamic category, add button, cart, login and OTP locators.
JavaScript Executor	Used for scroll, popup handling, and stable clicks on dynamic UI elements.
ActionChains	Used as fallback for product Add button click.
URL Assertions	Validates Health Monitors, cart, proceed/checkout state.
Page Object Model	Separates locators/page actions from tests and feature steps.
Data-Driven Testing	CSV files and scenario examples provide test data.
Allure Attachments	Screenshots and logs are attached for evidence.

13. Challenges and Solutions

Challenge	Problem	Solution
OTP Login	OTP is dynamic and cannot be hardcoded.	Automation waits while user manually enters OTP.
Dynamic Apollo UI	Buttons/classes change or render late.	Used robust XPath, text matching, JS click and retry logic.
Product not always added	Add button click sometimes did not update cart.	Added retry method and verified actual product name in cart.
Proceed button handling	Button text/class changed between runs.	Used visible text/title based click and validation for medicines-cart/address/payment states.
Report evidence	Need proof for pass/fail.	Captured screenshots and attached them in Allure.
BDD step mapping	English steps need Python implementation.	Implemented login_steps.py and shop_steps.py as step-definition layer.

14. Future Enhancements

- Add cross-browser execution for Firefox and Edge.
- Move more BDD test data from Examples to external CSV/Excel files.
- Add address and payment form validation scenarios in a safe test environment.
- Add Jenkins pipeline for CI/CD execution.
- Add parallel execution after stabilizing state-dependent cart scenarios.
- Add more negative test cases for invalid login, empty cart, and unavailable products.

15. Conclusion

The combined capstone project successfully demonstrates two automation framework styles for the same Apollo 247 workflow. ApolloAutomation shows a Selenium PyTest implementation with test files and PyTest fixtures. BBD_Apollo shows a Behave BDD implementation with Gherkin feature files, step definitions, and environment hooks. Both projects use Page Object Model, reusable utilities, logging, screenshots, Allure reporting, and validations for positive, negative, and end-to-end cases.

Attached Supporting Files

The screenshots have been removed from this Word documentation. The detailed Allure evidence and the converted project-content evidence are maintained as separate supporting PDF files listed below.

Supporting File	Purpose / Contents
CapstoneProject_Allure_Test_Report_ApolloAutomation_and_BBD_Apollo(1).pdf	Combined Allure Test Report for ApolloAutomation and BBD_Apollo. It summarizes the execution results for both projects, including 10 passed tests/scenarios per project and Allure evidence.
CapstoneProject_Zip_Converted_Project_Content(1).pdf	Converted project-content PDF from the CapstoneProject ZIP. It includes project structure, source code/config/data files, Allure summaries, execution logs, and extracted evidence files.

- Submit these two PDF files along with this documentation file as supporting evidence.
- The main documentation is now screenshot-free; the evidence is preserved in the supporting PDFs.

Combined Allure Test Report

ApolloAutomation Selenium PyTest + BBD_Apollo Behave BDD

Project	Framework	Total	Passed	Failed	Duration	Screenshots
ApolloAutomation	Selenium PyTest POM	**10**	**10**	0	3.4 min	10
BBD_Apollo	Behave BDD POM	**10**	**10**	0	3.3 min	10

Source data used: reports/allure-report widgets, reports/allure-report test-case JSON files, reports/screenshots images, and project configuration/test data files from the uploaded CapstoneProject.zip.

Overall result: Both projects show 10 total test cases/scenarios with 10 passed and 0 failed in the submitted Allure report data.

Project	Allure report source	Screenshot source
ApolloAutomation	ApolloAutomation/reports/allure-report	ApolloAutomation/reports/screenshots
BBD_Apollo	BBD_Apollo/reports/allure-report	BBD_Apollo/reports/screenshots

Generated from uploaded CapstoneProject.zip. This PDF is a static summary of the Allure report and test evidence.

ApolloAutomation - Allure Summary

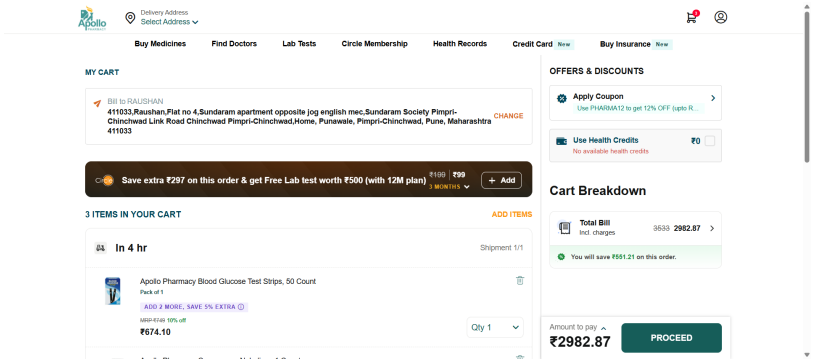
Framework	Selenium WebDriver + PyTest + Page Object Model	**Execution start**	22 May 2026, 11:10 AM
Total tests	10	**Execution stop**	22 May 2026, 11:14 AM
Passed	**10**	**Duration**	3.4 min
Failed / Broken / Skipped	0 / 0 / 0	**Screenshots captured**	10

Test definition files: tests/test_e2e.py, tests/test_positive_negative.py
Commands used: pytest -v -s; allure serve reports/allure-results

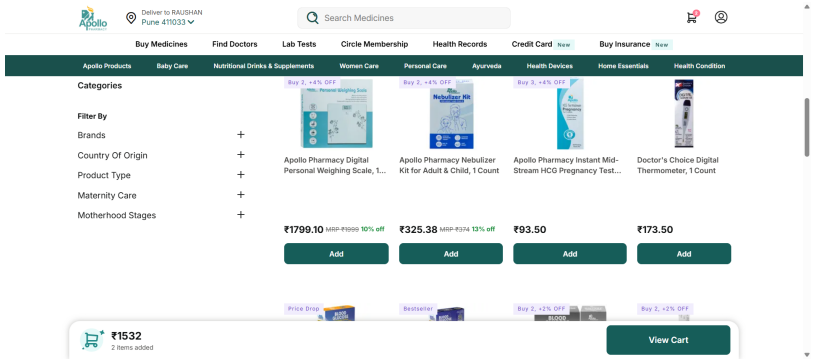
Test cases / scenarios from Allure

#	Name	Status	Duration	Suite / Feature	Attachments
1	test_login[data0]	**PASSED**	21.34 sec	test_e2e	0
2	test_shop_by_category_data[data0]	**PASSED**	1.82 sec	Shop By Category	0
3	test_shop_by_category_data[data1]	**PASSED**	1.69 sec	Shop By Category	0
4	test_shop_by_category_data[data2]	**PASSED**	1.46 sec	Shop By Category	0
5	test_shop_by_category_data[data3]	**PASSED**	4.22 sec	Shop By Category	0
6	test_shop_by_category_data[data4]	**PASSED**	3.83 sec	Shop By Category	0
7	test_apply_doctor_s_choice_filter_after_health_monitors	**PASSED**	38.75 sec	Health Monitors	0
8	test_click_health_monitors_category	**PASSED**	18.52 sec	Shop By Category	0
9	test_add_doctor_s_choice_product_to_cart	**PASSED**	49.74 sec	Cart	0
10	test_proceed_after_adding_product_to_cart	**PASSED**	1.0 min	Cart	0

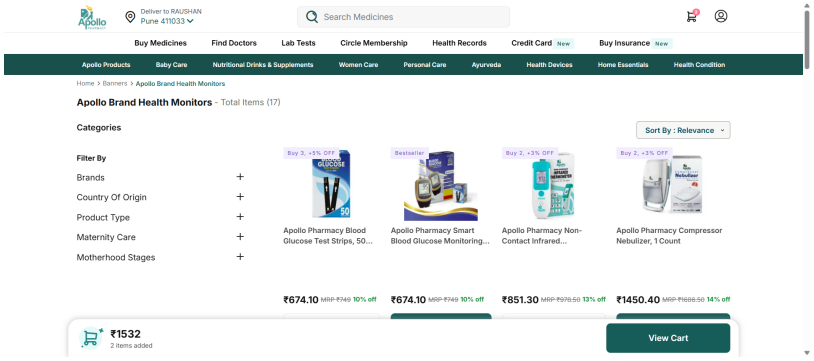
ApolloAutomation - E2E Evidence Screenshots



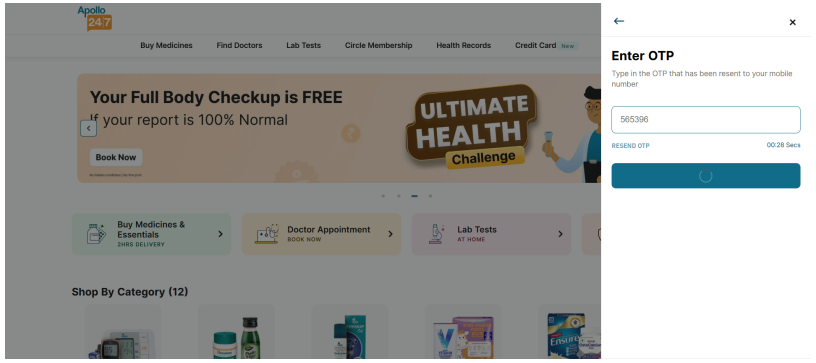
test add doctor s choice product to cart 20260522 164306



test apply doctor s choice filter after health monitors 20260522 164216

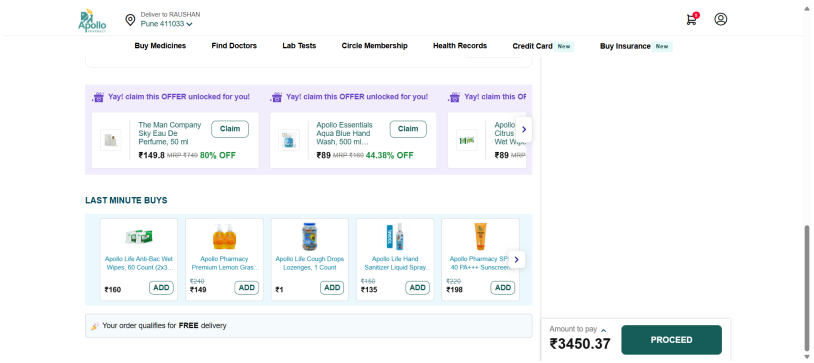


test click health monitors category 20260522 164137

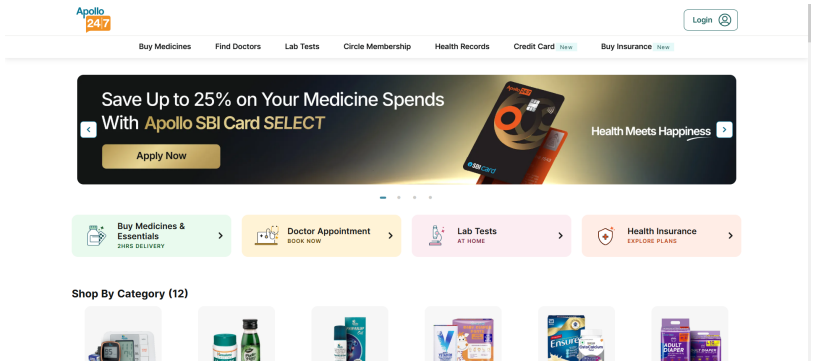


test login data0 20260522 164103

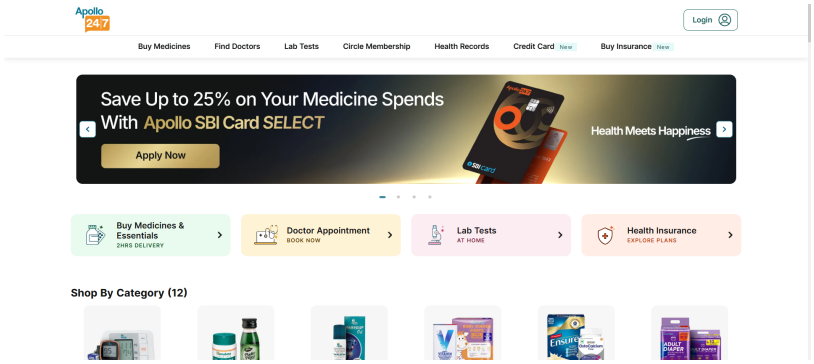
ApolloAutomation - Positive/Negative Evidence Screenshots



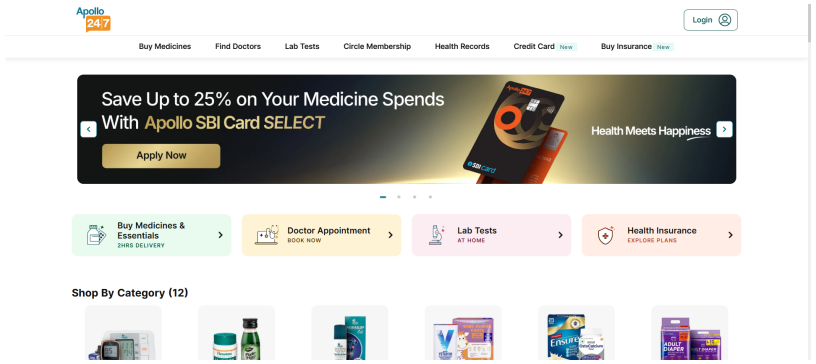
test proceed after adding product to cart 20260522 164409



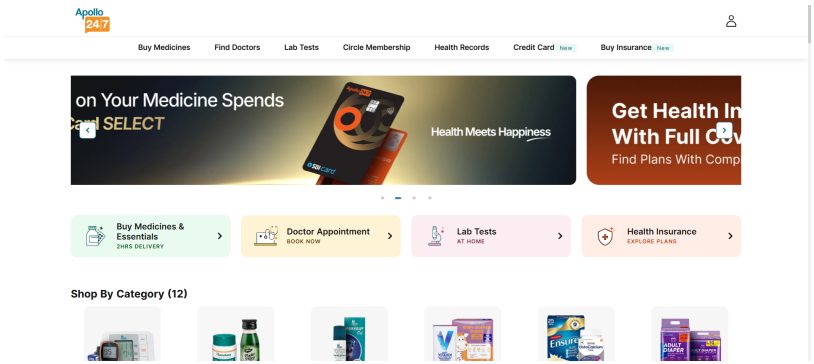
test shop by category data data0 20260522 164106



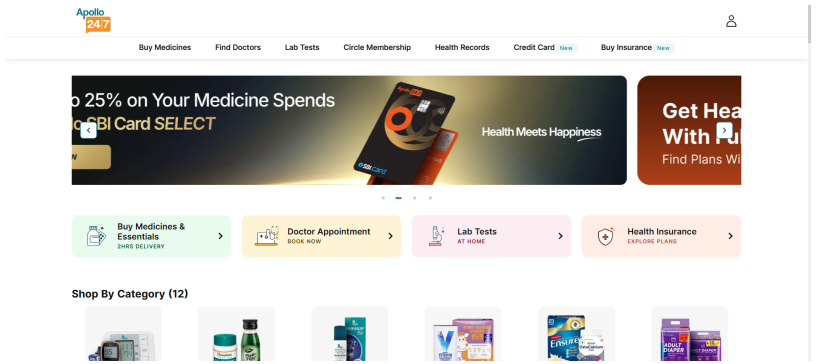
test shop by category data data1 20260522 164108



test shop by category data data2 20260522 164109



test shop by category data data3 20260522 164114



test shop by category data data4 20260522 164118

BBD_Apollo - Allure Summary

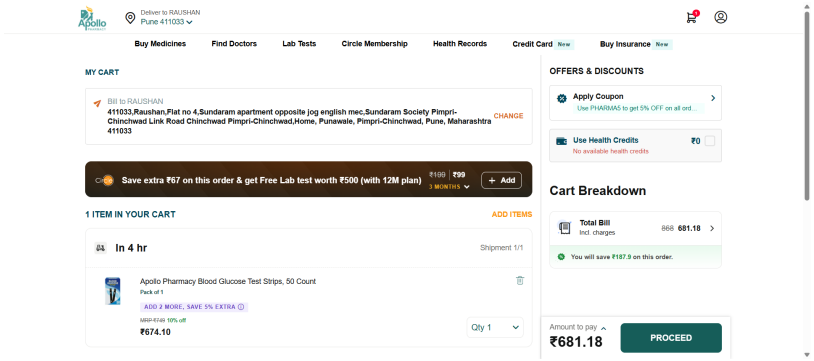
Framework	Selenium WebDriver + Behave BDD + Page Object Model	**Execution start**	23 May 2026, 06:41 AM
Total tests	10	**Execution stop**	23 May 2026, 06:44 AM
Passed	**10**	**Duration**	3.3 min
Failed / Broken / Skipped	0 / 0 / 0	**Screenshots captured**	10

Test definition files: features/login.feature, features/shop_by_category.feature, features/steps/*.py
Commands used: python runtests.py; allure open reports/allure-report

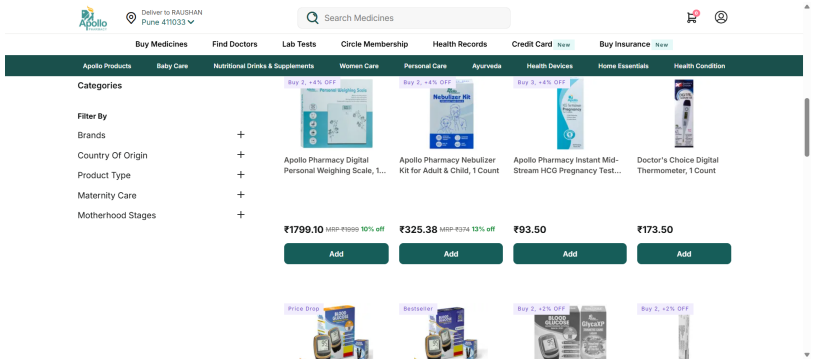
Test cases / scenarios from Allure

#	Name	Status	Duration	Suite / Feature	Attachments
1	Login with valid mobile number and OTP	**PASSED**	26.55 sec	Apollo login	0
2	Open Health Monitors category	**PASSED**	18.18 sec	Apollo shop by category	0
3	Apply Doctor S Choice filter	**PASSED**	34.50 sec	Apollo shop by category	0
4	Add Doctor S Choice product to cart	**PASSED**	46.01 sec	Apollo shop by category	0
5	Proceed after adding product to cart	**PASSED**	58.07 sec	Apollo shop by category	0
6	Validate category visibility -- @1.3	**PASSED**	1.61 sec	Apollo shop by category	0
7	Validate category visibility -- @1.4	**PASSED**	3.81 sec	Apollo shop by category	0
8	Validate category visibility -- @1.2	**PASSED**	2.13 sec	Apollo shop by category	0
9	Validate category visibility -- @1.1	**PASSED**	2.62 sec	Apollo shop by category	0
10	Validate category visibility -- @1.5	**PASSED**	3.80 sec	Apollo shop by category	0

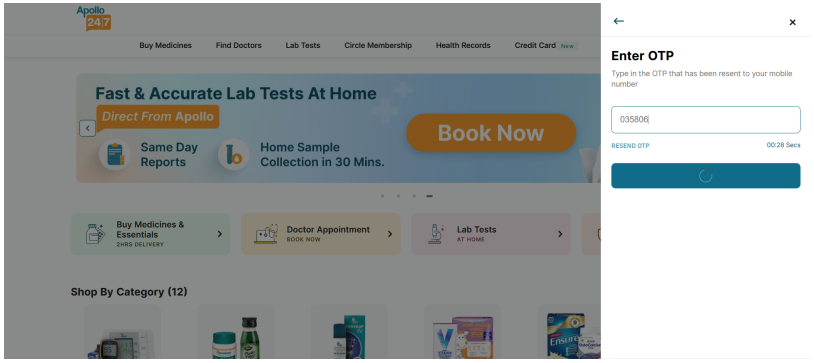
BBD_Apollo - E2E Evidence Screenshots



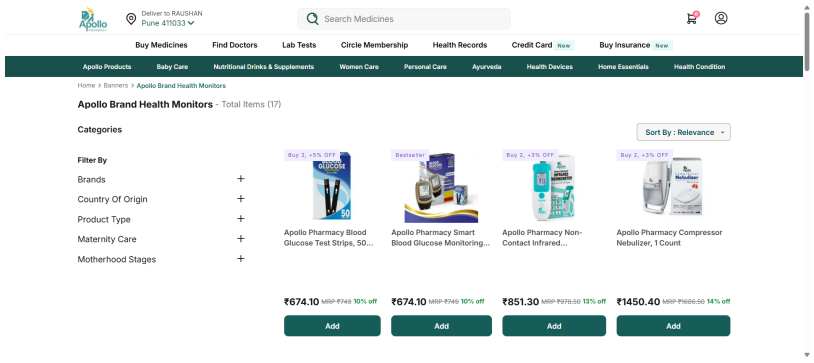
add doctor s choice product to cart 20260523 121334



apply doctor s choice filter 20260523 121248

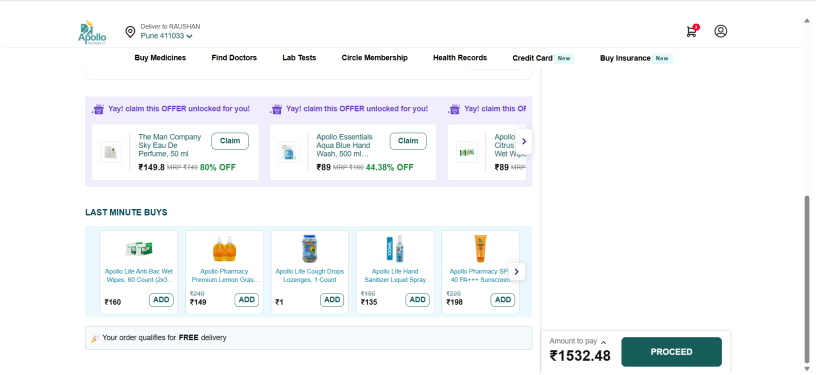


login with valid mobile number and otp 20260523 121142

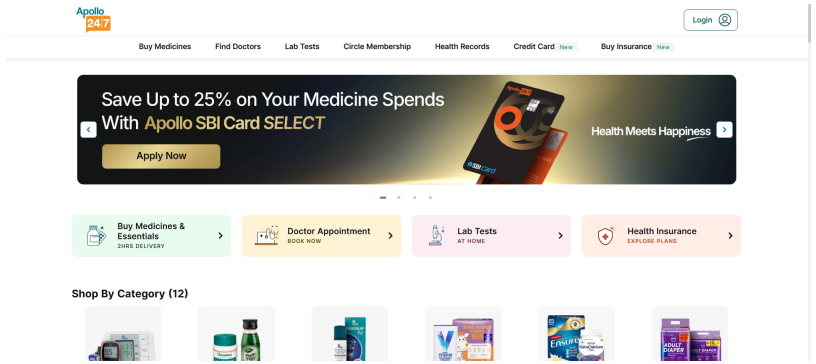


open health monitors category 20260523 121214

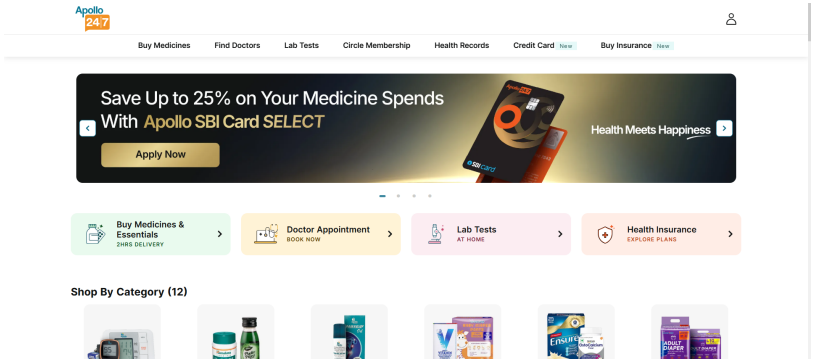
BBD_Apollo - Positive/Negative Evidence Screenshots



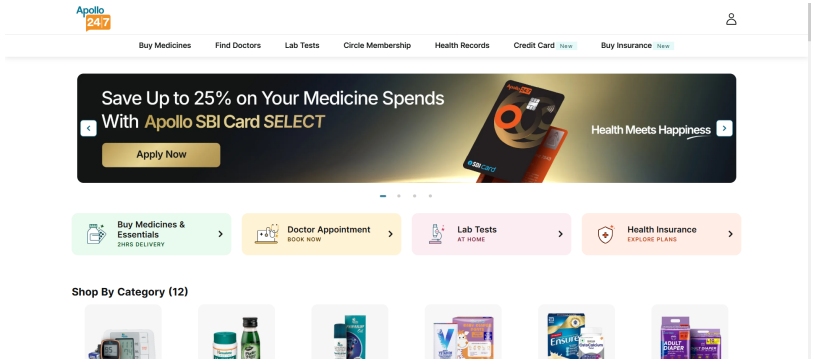
proceed after adding product to cart 20260523 121433



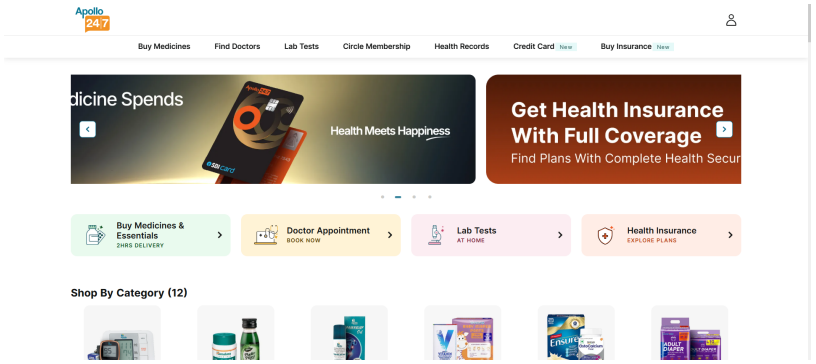
validate category visibility -- @1.1 20260523 121144



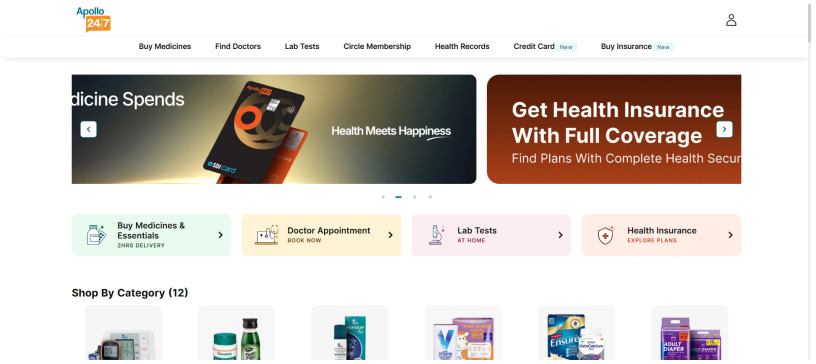
validate category visibility -- @1.2 20260523 121146



validate category visibility -- @1.3 20260523 121148



validate category visibility -- @1.4 20260523 121152



validate category visibility -- @1.5 20260523 121156

Test Data and Coverage Mapping

ApolloAutomation - Data files

data/login_data.csv

mobile_number	expected_result
8757726775	success

data/shop_by_category_data.csv

category_name	expected_result
Health Monitors	visible
Pain Relief	visible
Baby Care	visible
Fake Category	not_visible
Invalid Product	not_visible

BBD_Apollo - Data files

data/login_data.csv

mobile_number	expected_result
8757726775	success

data/shop_by_category_data.csv

category_name	expected_result
Health Monitors	visible
Pain Relief	visible
Baby Care	visible
Fake Category	not_visible
Invalid Product	not_visible

Coverage explanation

Positive cases: real categories such as Health Monitors, Pain Relief, and Baby Care are expected to be visible. Negative cases: Fake Category and Invalid Product are expected to be not visible. E2E coverage includes login, opening Health Monitors, applying Doctor S Choice filter, adding a product to cart, verifying cart contents, and proceeding from cart.

Project Artifact Notes

Section	Content taken from zip
Allure counts	reports/allure-report/widgets/summary.json
Test case names/status/duration	reports/allure-report/data/test-cases/*.json
Screenshots	reports/screenshots/*.png
Data files	data/login_data.csv and data/shop_by_category_data.csv
Logs available	logs/automation.log in each project

This report is generated directly from the uploaded CapstoneProject.zip content and is intended as a clean, printable test-report PDF for submission.

CapstoneProject(1).zip

Converted Project Content PDF

Includes source code, configuration, feature/test files, data files, Allure summaries, execution logs, and screenshots extracted from the uploaded ZIP.

Generated: 2026-05-23 18:17:25

Projects included: ApolloAutomation (Selenium PyTest) and BBD_Apollo (Behave BDD)

1. ZIP Conversion Summary

Item	Value
Source ZIP	CapstoneProject(1).zip
Total files in ZIP	7378
Files represented in this PDF	67
Excluded folders	.git, .venv, __pycache__, .pytest_cache, binary Allure assets
Reason for exclusions	Binary/environment folders are not useful in a PDF and make it unreadable. Source, data, reports, l

2. Allure Report Summaries

ApolloAutomation

Metric	Value
Total	10
Passed	10
Failed	0
Broken	0
Skipped	0
Duration	206.60 seconds

#	Test / Scenario	Status	Duration
1	test_shop_by_category_data[data0]	passed	1.82s
2	test_proceed_after_adding_product_to_cart	passed	62.70s
3	test_click_health_monitors_category	passed	18.52s
4	test_shop_by_category_data[data2]	passed	1.46s
5	test_login[data0]	passed	21.34s
6	test_shop_by_category_data[data3]	passed	4.22s
7	test_apply_doctor_s_choice_filter_after_health_monitors	passed	38.75s
8	test_shop_by_category_data[data4]	passed	3.83s
9	test_add_doctor_s_choice_product_to_cart	passed	49.74s
10	test_shop_by_category_data[data1]	passed	1.69s

BBD_Apollo

Metric	Value
Total	10
Passed	10
Failed	0
Broken	0
Skipped	0
Duration	197.28 seconds

#	Test / Scenario	Status	Duration
1	Apply Doctor S Choice filter	passed	34.50s
2	Validate category visibility -- @1.5	passed	3.80s
3	Validate category visibility -- @1.2	passed	2.13s
4	Proceed after adding product to cart	passed	58.07s
5	Validate category visibility -- @1.4	passed	3.81s
6	Validate category visibility -- @1.3	passed	1.61s
7	Validate category visibility -- @1.1	passed	2.62s
8	Login with valid mobile number and OTP	passed	26.55s
9	Open Health Monitors category	passed	18.18s
10	Add Doctor S Choice product to cart	passed	46.01s

3. Directory Tree

```
ApolloAutomation/.idea/.gitignore
ApolloAutomation/.idea/ApolloAutomation.iml
ApolloAutomation/.idea/inspectionProfiles/profiles_settings.xml
ApolloAutomation/.idea/misc.xml
ApolloAutomation/.idea/modules.xml
ApolloAutomation/.idea/vcs.xml
ApolloAutomation/.idea/workspace.xml
ApolloAutomation/config/config.properties
ApolloAutomation/conftest.py
ApolloAutomation/data/address_data.csv
ApolloAutomation/data/login_data.csv
ApolloAutomation/data/shop_by_category_data.csv
ApolloAutomation/logs/automation.log
ApolloAutomation/pages/basepage.py
ApolloAutomation/pages/loginpage.py
ApolloAutomation/pages/shopbycategorypage.py
ApolloAutomation/pages/__init__.py
ApolloAutomation/pytest.ini
ApolloAutomation/reports/allure-report/data/categories.csv
ApolloAutomation/reports/allure-report/data/categories.json
ApolloAutomation/reports/allure-report/data/suites.csv
ApolloAutomation/reports/allure-report/data/suites.json
ApolloAutomation/reports/allure-report/data/timeline.json
ApolloAutomation/reports/allure-report/export/influxDbData.txt
ApolloAutomation/reports/allure-report/export/mail.html
ApolloAutomation/reports/allure-report/export/prometheusData.txt
ApolloAutomation/reports/allure-report/history/categories-trend.json
ApolloAutomation/reports/allure-report/history/duration-trend.json
ApolloAutomation/reports/allure-report/history/history-trend.json
ApolloAutomation/reports/allure-report/history/history.json
ApolloAutomation/reports/allure-report/history/retry-trend.json
ApolloAutomation/reports/allure-report/index.html
ApolloAutomation/reports/allure-report/widgets/categories-trend.json
ApolloAutomation/reports/allure-report/widgets/categories.json
ApolloAutomation/reports/allure-report/widgets/duration-trend.json
ApolloAutomation/reports/allure-report/widgets/duration.json
ApolloAutomation/reports/allure-report/widgets/environment.json
ApolloAutomation/reports/allure-report/widgets/executors.json
ApolloAutomation/reports/allure-report/widgets/history-trend.json
ApolloAutomation/reports/allure-report/widgets/launch.json
ApolloAutomation/reports/allure-report/widgets/retry-trend.json
ApolloAutomation/reports/allure-report/widgets/severity.json
ApolloAutomation/reports/allure-report/widgets/status-chart.json
ApolloAutomation/reports/allure-report/widgets/suites.json
ApolloAutomation/reports/allure-report/widgets/summary.json
ApolloAutomation/reports/screenshots/passed_test_add_doctor_s_choice_product_to_cart_20260522_164306.png
ApolloAutomation/reports/screenshots/passed_test_apply_doctor_s_choice_filter_after_health_monitors_20260522_164216.png
ApolloAutomation/reports/screenshots/passed_test_click_health_monitors_category_20260522_164137.png
ApolloAutomation/reports/screenshots/passed_test_login_data0__20260522_164103.png
ApolloAutomation/reports/screenshots/passed_test_proceed_after_adding_product_to_cart_20260522_164409.png
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data0__20260522_164106.png
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data1__20260522_164108.png
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data2__20260522_164109.png
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data3__20260522_164114.png
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data4__20260522_164118.png
ApolloAutomation/requirements.txt
ApolloAutomation/tests/test_e2e.py
ApolloAutomation/tests/test_positive_negative.py
ApolloAutomation/tests/__init__.py
ApolloAutomation/utils/allure_report_generator.py
ApolloAutomation/utils/config_reader.py
ApolloAutomation/utils/csv_reader.py
ApolloAutomation/utils/excel_reader.py
ApolloAutomation/utils/logger.py
ApolloAutomation/utils/screenshot_util.py
BBD_Apollo/.idea/.gitignore
BBD_Apollo/.idea/BBD_Apollo.iml
BBD_Apollo/.idea/inspectionProfiles/profiles_settings.xml
BBD_Apollo/.idea/misc.xml
BBD_Apollo/.idea/modules.xml
BBD_Apollo/.idea/vcs.xml
BBD_Apollo/.idea/workspace.xml
BBD_Apollo/behave.ini
BBD_Apollo/config/config.ini
BBD_Apollo/data/login_data.csv
BBD_Apollo/data/shop_by_category_data.csv
BBD_Apollo/features/environment.py
BBD_Apollo/features/login.feature
BBD_Apollo/features/shop_by_category.feature
BBD_Apollo/features/steps/login_steps.py
```

BBD_Apollo/features/steps/shop_steps.py
BBD_Apollo/locators/login_locators.py
BBD_Apollo/locators/shop_locators.py
BBD_Apollo/logs/automation.log
BBD_Apollo/pages/base_page.py
BBD_Apollo/pages/login_page.py
BBD_Apollo/pages/shop_by_category_page.py
BBD_Apollo/pages/__init__.py
BBD_Apollo/pretty.output
BBD_Apollo/README.md
BBD_Apollo/reports/allure-report/data/categories.csv
BBD_Apollo/reports/allure-report/data/categories.json
BBD_Apollo/reports/allure-report/data/suites.csv
BBD_Apollo/reports/allure-report/data/suites.json
BBD_Apollo/reports/allure-report/data/timeline.json
BBD_Apollo/reports/allure-report/export/influxDbData.txt
BBD_Apollo/reports/allure-report/export/mail.html
BBD_Apollo/reports/allure-report/export/prometheusData.txt
BBD_Apollo/reports/allure-report/history/categories-trend.json
BBD_Apollo/reports/allure-report/history/duration-trend.json
BBD_Apollo/reports/allure-report/history/history-trend.json
BBD_Apollo/reports/allure-report/history/history.json
BBD_Apollo/reports/allure-report/history/retry-trend.json
BBD_Apollo/reports/allure-report/index.html
BBD_Apollo/reports/allure-report/widgets/categories-trend.json
BBD_Apollo/reports/allure-report/widgets/categories.json
BBD_Apollo/reports/allure-report/widgets/duration-trend.json
BBD_Apollo/reports/allure-report/widgets/duration.json
BBD_Apollo/reports/allure-report/widgets/environment.json
BBD_Apollo/reports/allure-report/widgets/executors.json
BBD_Apollo/reports/allure-report/widgets/history-trend.json
BBD_Apollo/reports/allure-report/widgets/launch.json
BBD_Apollo/reports/allure-report/widgets/retry-trend.json
BBD_Apollo/reports/allure-report/widgets/severity.json
BBD_Apollo/reports/allure-report/widgets/status-chart.json
BBD_Apollo/reports/allure-report/widgets/suites.json
BBD_Apollo/reports/allure-report/widgets/summary.json
BBD_Apollo/reports/screenshots/passed_add_doctor_s_choice_product_to_cart_20260523_121334.png
BBD_Apollo/reports/screenshots/passed_apply_doctor_s_choice_filter_20260523_121248.png
BBD_Apollo/reports/screenshots/passed_login_with_valid_mobile_number_and_otp_20260523_121142.png
BBD_Apollo/reports/screenshots/passed_open_health_monitors_category_20260523_121214.png
BBD_Apollo/reports/screenshots/passed_proceed_after_adding_product_to_cart_20260523_121433.png
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.1__20260523_121144.png
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.2__20260523_121146.png
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.3__20260523_121148.png
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.4__20260523_121152.png
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.5__20260523_121156.png
BBD_Apollo/requirements.txt
BBD_Apollo/runtests.py
BBD_Apollo/utils/config_reader.py
BBD_Apollo/utils/csv_reader.py
BBD_Apollo/utils/logger.py
BBD_Apollo/utils/screenshot_util.py
BBD_Apollo/utils/waits.py
BBD_Apollo/utils/__init__.py

4. Source Code, Config, Data, and Logs

ApolloAutomation

ApolloAutomation/config/config.properties

```
browser=chrome
base_url=https://www.apollo247.com/
implicit_wait=10
headless=false
timeout=20
```

ApolloAutomation/conftest.py

```
import pytest
import time
import allure

from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.chrome.options import Options as ChromeOptions
from webdriver_manager.chrome import ChromeDriverManager

from utils.config_reader import ConfigReader
from utils.logger import LogGen
from utils.screenshot_util import ScreenshotUtil
from utils.allure_report_generator import AllureReportGenerator

logger = LogGen.loggen()


@pytest.fixture(scope="session")
def driver():

    browser = ConfigReader.get("browser").strip().lower()

    logger.info("======================================")
    logger.info("Starting test session")
    logger.info("Reading Configuration")

    print(f"Browser from config: '{browser}'")

    logger.info(f"Browser from config : {browser}")

    base_url = ConfigReader.get("base_url").strip()

    logger.info(f"Base URL From Config : {base_url}")

    headless = ConfigReader.get("headless").strip().lower() == "true"

    logger.info(f"Headless : {headless}")

    if browser == "chrome":

        chrome_options = ChromeOptions()

        chrome_options.add_argument("--start-maximized")
        chrome_options.add_argument("--disable-notifications")
        chrome_options.add_argument("--disable-infobars")
        chrome_options.add_argument("--disable-extensions")

        if headless:
            chrome_options.add_argument("--headless=new")

        driver = webdriver.Chrome(
            service=ChromeService(
                ChromeDriverManager().install()
            ),
            options=chrome_options
        )

    else:

        raise Exception("Only Chrome browser is supported for this project")

    logger.info(f"Open Browser : {browser}")

    driver.get(base_url)

    logger.info(f"URL Loaded : {base_url}")

    yield driver

    logger.info("Waiting 6 seconds before closing browser")

    time.sleep(6)

    driver.quit()

    logger.info("Closing the browser")
    logger.info("Ending test session")
    logger.info("==========================================")
@pytest.hookimpl(hookwrapper=True)
def pytest_runtest_makereport(item, call):

    outcome = yield
    report = outcome.get_result()
```

```
    if report.when == "call":

        driver = item.funcargs.get("driver", None)

        if driver is not None:

            test_name = item.name

            if report.passed:

                logger.info(f"Test passed: {test_name}")

                screenshot_path = ScreenshotUtil.capture_screenshot(
                    driver,
                    f"passed_{test_name}"
                )

                with open(screenshot_path, "rb") as image_file:

                    allure.attach(
                        image_file.read(),
                        name=f"PASSED_{test_name}",
                        attachment_type=allure.attachment_type.PNG
                    )

            elif report.failed:

                logger.error(f"Test failed: {test_name}")

                screenshot_path = ScreenshotUtil.capture_screenshot(
                    driver,
                    f"failed_{test_name}"
                )

                with open(screenshot_path, "rb") as image_file:

                    allure.attach(
                        image_file.read(),
                        name=f"FAILED_{test_name}",
                        attachment_type=allure.attachment_type.PNG
                    )


def pytest_sessionfinish(session, exitstatus):

    logger.info("Test session finished")
    logger.info("Generating Allure report automatically")

    AllureReportGenerator.generate_report()
```

ApolloAutomation/data/address_data.csv

```
pincode,full_name,mobile_number,house_no,building_street,area_locality,address_type
411033,Raushan,6202615433,Flat no 4,Sundaram apartment opposite jog english mec,Sundaram Society Pimpri-Chinchwad Link Road Chinchwad
Pimpri-Chinchwad,Home
```

ApolloAutomation/data/login_data.csv

```
mobile_number,expected_result
8757726775,success
```

ApolloAutomation/data/shop_by_category_data.csv

```
category_name,expected_result
Health Monitors,visible
Pain Relief,visible
Baby Care,visible
Fake Category,not_visible
Invalid Product,not_visible
```

ApolloAutomation/pages/__init__.py

```
[empty file]
```

ApolloAutomation/pages/basepage.py

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.webdriver.common.by import By

from utils.logger import LogGen

logger = LogGen.loggen()


class BasePage:

    def __init__(self, driver, timeout=20):
        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    def get_element(self, locator, condition=EC.visibility_of_element_located, timeout=20):
        logger.info(f"Locating the element: {locator}")
        wait = WebDriverWait(self.driver, timeout)
        return wait.until(condition(locator))

    def click(self, locator, timeout=10):
        logger.info(f"Clicking the element: {locator}")
        self.get_element(locator, EC.element_to_be_clickable, timeout).click()
```

```
    def type(self, locator, text, timeout=10):
        logger.info(f"Typing into element: {locator}")
        element = self.get_element(locator, EC.visibility_of_element_located, timeout)
        element.clear()
        element.send_keys(text)

    def is_visible(self, locator, timeout=5):
        logger.info(f"Checking visibility of element: {locator}")

        try:
            return self.get_element(
                locator,
                EC.visibility_of_element_located,
                timeout
            ).is_displayed()
        except:
            return False

    def scroll_to_element(self, locator, timeout=10):
        logger.info(f"Scrolling to element: {locator}")

        element = self.get_element(
            locator,
            EC.visibility_of_element_located,
            timeout
        )

        self.driver.execute_script(
            "arguments[0].scrollIntoView({block:'center'});",
            element
        )

    def js_click(self, locator, timeout=10):
        logger.info(f"Clicking element using JavaScript: {locator}")

        element = self.get_element(
            locator,
            EC.visibility_of_element_located,
            timeout
        )

        self.driver.execute_script(
            "arguments[0].click();",
            element
        )

    def close_popup_if_present(self):
        logger.info("Checking popup on page")

        self.driver.execute_script("""
            const popup = document.querySelector('ct-web-popup-imageonly');
            if (popup) {
                popup.remove();
            }

            const closeButtons = document.querySelectorAll(
                '[class*="close"], [aria-label*="close"], [aria-label*="Close"]'
            );

            closeButtons.forEach(button => {
                try {
                    button.click();
                } catch(e) {}
            });
        """)
```

ApolloAutomation/pages/loginpage.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.common.keys import Keys
from selenium.webdriver.support.ui import WebDriverWait
from selenium.common.exceptions import StaleElementReferenceException
import time

from pages.basepage import BasePage
from utils.logger import LogGen

logger = LogGen.loggen()


class LoginPage(BasePage):

    LOGIN_ICON = (
        By.XPATH,
        "//*[contains(text(),'Login') or contains(text(),'Sign in') or contains(text(),'Sign In')]"
    )

    MOBILE_INPUT = (
        By.XPATH,
        "//input"
    )

    CONTINUE_BUTTON = (
        By.XPATH,
        "//button[contains(text(),'Continue')]"
    )

    OTP_SCREEN_TEXT = (
        By.XPATH,
        "//*[contains(text(),'OTP') or contains(text(),'otp') or contains(text(),'Resend') or contains(text(),'sent')]"
    )
```

```
OTP_INPUTS = (
    By.XPATH,
    "//input"
)

VERIFY_BUTTON = (
    By.XPATH,
    "//button[contains(text(),'Verify') or contains(text(),'Continue') or contains(text(),'Submit')]"
)

def open_login_popup(self):

    logger.info("Opening login popup")

    self.close_popup_if_present()

    time.sleep(2)

    try:
        self.click(self.LOGIN_ICON, timeout=8)
        logger.info("Login clicked using normal Selenium click")
        time.sleep(2)
        return

    except Exception as e:
        logger.info(f"Normal login click failed: {e}")

    try:
        self.js_click(self.LOGIN_ICON, timeout=8)
        logger.info("Login clicked using JavaScript click")
        time.sleep(2)
        return

    except Exception as e:
        logger.info(f"JavaScript login click failed: {e}")

    clicked = self.driver.execute_script(
        """
        const elements = document.querySelectorAll(
            'button, div, span, p, a'
        );

        for (const element of elements) {
            const text = (element.innerText || element.textContent || '')
                .trim()
                .toLowerCase();

            if (
                text === 'login' ||
                text.includes('login') ||
                text.includes('sign in')
            ) {
                element.scrollIntoView({block: 'center'});
                element.click();
                return true;
            }
        }

        return false;
        """
    )

    if clicked:
        logger.info("Login clicked using JavaScript text search")
        time.sleep(2)
        return

    raise Exception("Login button not found on Apollo247 homepage")

def login_with_mobile_number(self, mobile_number):

    logger.info("Starting login with mobile number")

    self.open_login_popup()

    logger.info(f"Entering mobile number automatically: {mobile_number}")

    mobile_box = self.get_element(self.MOBILE_INPUT, timeout=10)

    mobile_box.clear()

    mobile_box.send_keys(mobile_number)

    time.sleep(1)

    logger.info("Clicking Continue button")

    self.click(self.CONTINUE_BUTTON, timeout=10)

    time.sleep(3)

def is_otp_screen_visible(self):

    logger.info("Checking OTP screen")

    return self.is_visible(self.OTP_SCREEN_TEXT, timeout=10)

def wait_for_otp_entry_and_submit(self, mobile_number, timeout=60):

    logger.info("Waiting for OTP entry")
    wait = WebDriverWait(
```

```
            self.driver,
            timeout,
            ignored_exceptions=(StaleElementReferenceException,)
        )

        def otp_entered(driver):

            try:
                inputs = driver.find_elements(*self.OTP_INPUTS)

                visible_values = []

                for input_box in inputs:
                    try:
                        if input_box.is_displayed():
                            value = input_box.get_attribute("value")

                            if value:
                                visible_values.append(value)

                    except StaleElementReferenceException:
                        return False

                entered_text = "".join(visible_values).strip()

                if entered_text == mobile_number:
                    return False

                if len(entered_text) >= 4:
                    return True

                return False

            except StaleElementReferenceException:
                return False

        wait.until(otp_entered)

        logger.info("OTP entered manually")

        self.submit_otp()

        time.sleep(2)

        return True

    def submit_otp(self):

        logger.info("Submitting OTP")

        try:
            buttons = self.driver.find_elements(*self.VERIFY_BUTTON)

            for button in buttons:
                try:
                    if button.is_displayed() and button.is_enabled():
                        self.driver.execute_script(
                            "arguments[0].click();",
                            button
                        )

                        logger.info("OTP submitted using button")

                        return

                except StaleElementReferenceException:
                    continue

        except Exception as e:
            logger.info(f"OTP submit button not clicked: {e}")

        try:
            inputs = self.driver.find_elements(*self.OTP_INPUTS)

            for input_box in inputs:
                try:
                    if input_box.is_displayed():
                        input_box.send_keys(Keys.ENTER)

                        logger.info("OTP submitted using Enter key")

                        return

                except StaleElementReferenceException:
                    continue

        except Exception as e:
            logger.info(f"OTP Enter key submit failed: {e}")
```

ApolloAutomation/pages/shopbycategorypage.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.common.action_chains import ActionChains
import time

from pages.basepage import BasePage
from utils.logger import LogGen

logger = LogGen.loggen()
class ShopByCategoryPage(BasePage):
```

```
    HEALTH_MONITORS_TEXT = (
        By.XPATH,
        "//*[normalize-space()='Health Monitors']"
    )

    HEALTH_MONITORS_URL = (
        "https://www.apollopharmacy.in/shop-by-category/apollo-brand-health-monitors"
    )

    EXACT_ADD_BUTTON = (
        By.XPATH,
        "(//button[@aria-label='Add' and .//span[normalize-space()='Add']])[1]"
    )

    CART_ICON = (
        By.XPATH,
        "//*[contains(@href,'cart') or contains(@class,'cart') or contains(text(),'Cart')]"
    )

    CART_TEXT = (
        By.XPATH,
        "//*[contains(text(),'Cart') or contains(text(),'cart')]"
    )

    EMPTY_CART_TEXT = (
        By.XPATH,
        "//*[contains(text(),'YOUR CART IS EMPTY') or contains(text(),'Your cart is empty') or contains(text(),'empty') or
contains(text(),'Empty')]"
    )

    PROCEED_BUTTON = (
        By.XPATH,
        "//button[@title='Proceed' and @aria-label='Button' and .//span[normalize-space()='Proceed']]"
    )

    def is_category_visible(self, category_name, timeout=5):

        self.close_popup_if_present()

        locator = (
            By.XPATH,
            f"//*[normalize-space()='{category_name}']"
        )

        return self.is_visible(locator, timeout=timeout)

    def click_health_monitors(self):

        logger.info("Opening Health Monitors category")

        self.close_popup_if_present()

        try:
            self.scroll_to_element(self.HEALTH_MONITORS_TEXT, timeout=5)

            time.sleep(1)

            health_monitor_element = self.get_element(
                self.HEALTH_MONITORS_TEXT,
                timeout=5
            )

            self.driver.execute_script(
                "arguments[0].click();",
                health_monitor_element
            )

            WebDriverWait(self.driver, 5).until(
                lambda driver:
                "apollo-brand-health-monitors" in driver.current_url
            )

        except Exception as e:

            logger.info(f"Health Monitors click failed: {e}")
            logger.info("Opening Health Monitors page directly")

            self.driver.get(self.HEALTH_MONITORS_URL)

            WebDriverWait(self.driver, 10).until(
                lambda driver:
                "apollo-brand-health-monitors" in driver.current_url
            )

        time.sleep(6)

    def is_health_monitors_page_opened(self):

        logger.info("Validating Health Monitors page")

        current_url = self.driver.current_url

        logger.info(f"Current URL is: {current_url}")

        return "apollo-brand-health-monitors" in current_url

    def is_brands_filter_visible(self):

        logger.info("Checking Brands filter visibility")

        time.sleep(5)
```

```
    page_text = self.driver.find_element(By.TAG_NAME, "body").text.lower()

    if "brands" in page_text:
        logger.info("Brands filter text found on page")
        return True

    logger.error("Brands filter text not found on page")
    return False

def open_brands_filter(self):

    logger.info("Opening Brands filter")

    self.close_popup_if_present()

    time.sleep(3)

    opened = self.driver.execute_script(
        """
        const elements = Array.from(
            document.querySelectorAll('label, div, span, h3, button')
        );

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = (
                element.innerText ||
                element.textContent ||
                ''

            ).trim().toLowerCase();

            if (text === 'brands' || text.includes('brands')) {

                element.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                const clickable =
                    element.closest('label, button, div') || element;

                clickable.click();

                return true;
            }
        }

        return false;
        """
    )

    assert opened is True, \
        "Brands filter was not found or not clicked"

    logger.info("Brands filter opened successfully")

    time.sleep(3)

def apply_doctor_s_choice_filter(self):

    logger.info("Applying Doctor S Choice brand filter")

    self.close_popup_if_present()

    self.open_brands_filter()

    time.sleep(3)

    clicked = self.driver.execute_script(
        """
        function normalizeText(value) {
            return (value || '')
                .replace(/['']/g, '')
                .replace(/[^a-zA-Z0-9]/g, ' ')
                .replace(/\\s+/g, ' ')
                .trim()
                .toLowerCase();
        }

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
```

```
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        // Scroll a little inside the page to load/open brand options properly
        window.scrollBy(0, 300);

        const elements = Array.from(
            document.querySelectorAll('label, div, span, p, h3, button')
        );

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = normalizeText(
                element.innerText || element.textContent
            );

            /*
                Matches:
                Doctor S Choice
                Doctors Choice
                Doctor's Choice
                doctor s choice
            */
            const isDoctorChoice =
                text.includes('doctor s choice') ||
                text.includes('doctors choice') ||
                (
                    text.includes('doctor') &&
                    text.includes('choice')
                );

            if (isDoctorChoice) {

                element.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                const clickable =
                    element.closest('label, button, div') || element;

                clickable.dispatchEvent(
                    new MouseEvent('mouseover', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('mousedown', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('mouseup', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('click', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                return true;
            }
        }

        return false;
        """
    )

    assert clicked is True, \
        "Doctor S Choice filter option was not found or not clicked"

    logger.info("Doctor S Choice filter clicked successfully")

    time.sleep(5)

def get_exact_add_button(self):

    logger.info("Finding exact Add button")
    add_button = self.get_element(
```

```
            self.EXACT_ADD_BUTTON,
            timeout=20
        )
        self.driver.execute_script(
            "arguments[0].scrollIntoView({block:'center'});",
            add_button
        )

        time.sleep(2)

        return add_button

    def get_product_name_from_add_button(self, add_button):

        logger.info("Getting product name from selected Add button")

        product_name = self.driver.execute_script(
            """
            const addBtn = arguments[0];

            let parent = addBtn;

            for (let i = 0; i < 12; i++) {

                if (!parent) {
                    break;
                }

                const text = (parent.innerText || parent.textContent || '').trim();

                if (text.length > 10 && text.toLowerCase().includes('add')) {

                    const lines = text
                        .split('\\n')
                        .map(line => line.trim())
                        .filter(line => line.length > 0);

                    for (const line of lines) {

                        const lower = line.toLowerCase();

                        if (
                            !lower.includes('add') &&
                            !lower.includes('rs') &&
                            !lower.includes('mrp') &&
                            !lower.includes('off') &&
                            !lower.includes('%') &&
                            !lower.includes('rating') &&
                            !lower.includes('cart') &&
                            !lower.includes('bestseller') &&
                            line.length > 5
                        ) {
                            return line;
                        }
                    }
                }

                parent = parent.parentElement;
            }

            return '';
            """,
            add_button
        )

        product_name = product_name.strip()

        logger.info(f"Selected product name: {product_name}")

        if product_name == "":
            raise Exception("Product name could not be captured")

        return product_name

    def click_exact_add_button_once(self, add_button):

        logger.info("Clicking exact Add button one time")

        self.driver.execute_script(
            "arguments[0].scrollIntoView({block:'center'});",
            add_button
        )
        time.sleep(1)

        button_text = add_button.text.strip()

        logger.info(f"Add button text before click: {button_text}")

        assert button_text.lower() == "add", \
            "Add button is not available"

        try:
            add_button.click()
            logger.info("Clicked Add using normal Selenium click")
            return

        except Exception as e:
            logger.info(f"Normal Selenium click failed: {e}")

        try:
```

```
            ActionChains(self.driver)\
                .move_to_element(add_button)\
                .pause(1)\
                .click(add_button)\
                .perform()

            logger.info("Clicked Add using ActionChains")
            return

        except Exception as e:
            logger.info(f"ActionChains click failed: {e}")

        self.driver.execute_script(
            """
            const button = arguments[0];

            button.scrollIntoView({block:'center'});

            button.dispatchEvent(new MouseEvent('mouseover', {bubbles: true}));
            button.dispatchEvent(new MouseEvent('mousedown', {bubbles: true}));
            button.dispatchEvent(new MouseEvent('mouseup', {bubbles: true}));
            button.dispatchEvent(new MouseEvent('click', {bubbles: true}));
            """,
            add_button
        )

        logger.info("Clicked Add using JavaScript mouse events")

    def go_to_cart(self):

        logger.info("Opening cart")

        self.close_popup_if_present()

        try:
            self.click(self.CART_ICON, timeout=10)

        except Exception:
            logger.info("Cart icon click failed, opening cart directly")
            self.driver.get("https://www.apollopharmacy.in/cart")

        WebDriverWait(self.driver, 15).until(
            lambda driver:
            "cart" in driver.current_url.lower()
            or len(driver.find_elements(*self.CART_TEXT)) > 0
        )

        time.sleep(5)

    def is_cart_page_opened(self):

        logger.info("Checking cart page")

        return (
            "cart" in self.driver.current_url.lower()
            or self.is_visible(self.CART_TEXT, timeout=5)
        )

    def is_cart_empty(self):

        logger.info("Checking whether cart is empty")

        empty_items = self.driver.find_elements(*self.EMPTY_CART_TEXT)

        for item in empty_items:

            try:
                if item.is_displayed():
                    logger.error("Cart is empty")
                    return True

            except Exception:
                continue

        return False

    def is_exact_product_present_in_cart(self, product_name):

        logger.info(f"Checking exact product in cart: {product_name}")

        time.sleep(3)

        if self.is_cart_empty():
            return False

        cart_text = self.driver.find_element(By.TAG_NAME, "body").text

        if product_name.lower() in cart_text.lower():
            logger.info("Exact product found in cart")
            return True

        logger.error("Exact product not found in cart")
        return False

    def add_exact_product_to_cart_with_retry(self, max_attempts=3):

        logger.info("Adding exact product to cart with retry")

        product_name = None

        for attempt in range(1, max_attempts + 1):
```

```
        logger.info(f"Add product attempt: {attempt}")

        self.driver.get(self.HEALTH_MONITORS_URL)

        WebDriverWait(self.driver, 10).until(
            lambda driver:
            "apollo-brand-health-monitors" in driver.current_url
        )

        time.sleep(6)

        self.apply_doctor_s_choice_filter()

        add_button = self.get_exact_add_button()

        product_name = self.get_product_name_from_add_button(add_button)

        logger.info(f"Trying to add product: {product_name}")

        self.click_exact_add_button_once(add_button)

        time.sleep(7)

        self.go_to_cart()

        if (
            self.is_cart_page_opened()
            and self.is_exact_product_present_in_cart(product_name)
        ):
            logger.info("Product added and verified in cart successfully")
            return product_name

        logger.error("Product not found in cart after Add click")

    raise AssertionError(
        f"Product was not added to cart after {max_attempts} attempts"
    )

def click_proceed_button(self):
    logger.info("Clicking Proceed button on cart page")

    assert self.is_cart_page_opened(), \
        "Cart page is not opened, cannot click Proceed"

    assert not self.is_cart_empty(), \
        "Cart is empty, cannot click Proceed"

    self.driver.execute_script(
        "window.scrollTo(0, document.body.scrollHeight);"
    )

    time.sleep(3)

    clicked = self.driver.execute_script(
        """
        const elements = Array.from(
            document.querySelectorAll('button, span, div')
        );

        function normalizeText(value) {
            return (value || '')
                .replace(/\\s+/g, ' ')
                .trim()
                .toLowerCase();
        }

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = normalizeText(
                element.innerText || element.textContent
            );

            const title = normalizeText(
                element.getAttribute('title')
            );

            if (
                text === 'proceed' ||
                title === 'proceed'
            ) {

                let clickable = element.closest('button');

                if (!clickable) {
```

```
                    clickable = element;
                }

                clickable.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                clickable.dispatchEvent(
                    new MouseEvent('mouseover', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('mousedown', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('mouseup', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                clickable.dispatchEvent(
                    new MouseEvent('click', {
                        bubbles: true,
                        cancelable: true,
                        view: window
                    })
                );

                return true;
            }
        }

        return false;
        """
    )

    assert clicked is True, \
        "Proceed button was not found or not clicked"

    logger.info("Proceed button clicked successfully")

    time.sleep(5)

def is_after_proceed_page_opened(self):

    logger.info("Checking page after clicking Proceed")

    time.sleep(5)

    page_text = self.driver.find_element(By.TAG_NAME, "body").text.lower()
    current_url = self.driver.current_url.lower()

    logger.info(f"Current URL after proceed: {current_url}")

    return (
        "medicines-cart" in current_url
        or "select address" in page_text
        or "delivery address" in page_text
        or "add address" in page_text
        or "address" in page_text
        or "payment" in page_text
        or "login" in page_text
        or "continue" in page_text
        or "checkout" in current_url
        or "address" in current_url
        or "payment" in current_url
    )
```

ApolloAutomation/pytest.ini

```
[pytest]
testpaths = tests
python_files = test_*.py
python_functions = test_*
addopts = -v -s --alluredir=reports/allure-results
markers =
    order: mark test execution order
```

ApolloAutomation/requirements.txt

```
selenium
webdriver-manager
pytest
pytest-order
openpyxl
allure-pytest
pytest-html
```

ApolloAutomation/tests/__init__.py

[empty file]

ApolloAutomation/tests/test_e2e.py

```
import pytest
import allure

from pages.loginpage import LoginPage
from pages.shopbycategorypage import ShopByCategoryPage
from utils.csv_reader import CSVReader
from utils.config_reader import ConfigReader
from utils.logger import LogGen

logger = LogGen.loggen()


@pytest.mark.order(1)
@pytest.mark.parametrize(
    "data",
    CSVReader.read_csv("login_data.csv")
)
def test_login(driver, data):

    driver.get(ConfigReader.get("base_url"))

    login_page = LoginPage(driver)

    mobile_number = data["mobile_number"]

    logger.info(f"Trying login with mobile number: {mobile_number}")

    login_page.login_with_mobile_number(mobile_number)

    assert login_page.is_otp_screen_visible(), \
        "OTP screen should be visible after entering mobile number"

    print("\nEnter OTP manually in browser.")
    print("Do not press anything in terminal.")
    print("Automation will continue automatically after OTP is entered.")
    print("Maximum wait time is 60 seconds.\n")

    otp_submitted = login_page.wait_for_otp_entry_and_submit(
        mobile_number,
        timeout=30
    )

    assert otp_submitted is True, \
        "OTP was not entered or submitted"

    logger.info("OTP entered and submitted successfully")


@allure.epic("Apollo247 Automation")
@allure.feature("Shop By Category")
@allure.story("Open Health Monitors")
@pytest.mark.order(3)
def test_click_health_monitors_category(driver):

    driver.get(ConfigReader.get("base_url"))

    shop_by_category_page = ShopByCategoryPage(driver)

    shop_by_category_page.click_health_monitors()

    assert shop_by_category_page.is_health_monitors_page_opened(), \
        "Health Monitors page did not open"


@allure.epic("Apollo247 Automation")
@allure.feature("Health Monitors")
@allure.story("Apply Doctor S Choice Filter")
@pytest.mark.order(4)
def test_apply_doctor_s_choice_filter_after_health_monitors(driver):

    driver.get(ConfigReader.get("base_url"))

    shop_by_category_page = ShopByCategoryPage(driver)

    shop_by_category_page.click_health_monitors()

    assert shop_by_category_page.is_health_monitors_page_opened(), \
        "Health Monitors page did not open"

    assert shop_by_category_page.is_brands_filter_visible(), \

        "Brands filter is not visible"

    shop_by_category_page.apply_doctor_s_choice_filter()

    assert shop_by_category_page.is_health_monitors_page_opened(), \
        "Health Monitors page is not opened after applying Doctor S Choice filter"


@allure.epic("Apollo247 Automation")
@allure.feature("Cart")
@allure.story("Add Exact Doctor S Choice Product To Cart")
@pytest.mark.order(5)
def test_add_doctor_s_choice_product_to_cart(driver):
    driver.get(ConfigReader.get("base_url"))
```

```
    shop_by_category_page = ShopByCategoryPage(driver)

    product_name = shop_by_category_page.add_exact_product_to_cart_with_retry(
        max_attempts=3
    )

    assert product_name is not None, \
        "Product name was not captured"

    assert shop_by_category_page.is_cart_page_opened(), \
        "Cart page did not open"

    assert shop_by_category_page.is_exact_product_present_in_cart(product_name), \
        f"Expected product not found in cart: {product_name}"


@allure.epic("Apollo247 Automation")
@allure.feature("Cart")
@allure.story("Proceed After Adding Product To Cart")
@pytest.mark.order(6)
def test_proceed_after_adding_product_to_cart(driver):

    driver.get(ConfigReader.get("base_url"))

    shop_by_category_page = ShopByCategoryPage(driver)

    product_name = shop_by_category_page.add_exact_product_to_cart_with_retry(
        max_attempts=3
    )

    assert product_name is not None, \
        "Product name was not captured"

    assert shop_by_category_page.is_cart_page_opened(), \
        "Cart page did not open"

    assert shop_by_category_page.is_exact_product_present_in_cart(product_name), \
        f"Expected product not found in cart: {product_name}"

    shop_by_category_page.click_proceed_button()

    assert shop_by_category_page.is_after_proceed_page_opened(), \
        "Page did not move forward after clicking Proceed"
```

ApolloAutomation/tests/test_positive_negative.py

```
import pytest
import allure

from pages.shopbycategorypage import ShopByCategoryPage
from utils.csv_reader import CSVReader
from utils.config_reader import ConfigReader


@allure.epic("Apollo247 Automation")
@allure.feature("Shop By Category")
@pytest.mark.parametrize(
    "data",
    CSVReader.read_csv("shop_by_category_data.csv")
)
@pytest.mark.order(2)
def test_shop_by_category_data(driver, data):

    driver.get(ConfigReader.get("base_url"))

    shop_by_category_page = ShopByCategoryPage(driver)

    category_name = data["category_name"]
    expected_result = data["expected_result"]

    if expected_result == "visible":

        actual_result = shop_by_category_page.is_category_visible(
            category_name,
            timeout=5
        )

        assert actual_result is True, \
            f"{category_name} category should be visible"

    elif expected_result == "not_visible":

        actual_result = shop_by_category_page.is_category_visible(
            category_name,
            timeout=2
        )

        assert actual_result is False, \
            f"{category_name} category should not be visible"
```

ApolloAutomation/utils/allure_report_generator.py

```
import os
import subprocess

from utils.logger import LogGen

logger = LogGen.loggen()
class AllureReportGenerator:
    @staticmethod
```

```
    def generate_report():

        try:
            logger.info("Starting automatic Allure report generation")

            allure_results_dir = os.path.join(
                "reports",
                "allure-results"
            )

            allure_report_dir = os.path.join(
                "reports",
                "allure-report"
            )

            if not os.path.exists(allure_results_dir):

                logger.error("Allure results directory not found")

                print("\n====================================")
                print("Allure results directory not found")
                print("====================================\n")

                return

            command = [
                "allure",
                "generate",
                allure_results_dir,
                "-o",
                allure_report_dir,
                "--clean"
            ]

            logger.info(f"Running command: {' '.join(command)}")

            result = subprocess.run(
                command,
                capture_output=True,
                text=True,
                shell=True
            )

            if result.returncode == 0:

                logger.info("Allure report generated successfully")

                print("\n====================================")
                print("Allure report generated successfully")
                print(f"Report location: {allure_report_dir}")
                print("====================================\n")

            else:

                logger.error("Failed to generate Allure report")
                logger.error(result.stderr)

                print("\n====================================")
                print("Failed to generate Allure report")
                print(result.stderr)
                print("====================================\n")

        except Exception as e:

            logger.error("Exception occurred while generating Allure report")
            logger.error(str(e))

            print("\n====================================")
            print("Allure report generation failed")
            print(str(e))
            print("====================================\n")
```

ApolloAutomation/utils/config_reader.py

```
import os


class ConfigReader:
    _config = {}

    @classmethod
    def load_config(cls):
        config_path = os.path.join(
            os.path.dirname(os.path.dirname(__file__)),
            "config",
            "config.properties"
        )

        cls._config = {}

        with open(config_path, "r") as configfile:
            for line in configfile:
                line = line.strip()

                if line and not line.startswith("#"):
                    key, value = line.split("=", 1)
                    cls._config[key.strip()] = value.strip()

        return cls._config
    @classmethod
```

```
    def get(cls, key):
        return cls.load_config().get(key)

    @classmethod
    def get_base_url(cls):
        return cls.get("base_url")

    @classmethod
    def get_browser(cls):
        return cls.get("browser")

    @classmethod
    def get_implicit_wait(cls):
        return int(cls.get("implicit_wait"))

    @classmethod
    def get_timeout(cls):
        return int(cls.get("timeout"))

    @classmethod
    def is_headless(cls):
        return cls.get("headless").lower() == "true"
```

ApolloAutomation/utils/csv_reader.py

```
import csv
import os


class CSVReader:

    @staticmethod
    def read_csv(file_name):
        data = []

        base_dir = os.path.dirname(os.path.dirname(__file__))
        file_path = os.path.join(base_dir, "data", file_name)

        with open(file_path, newline='', encoding="utf-8") as csvfile:
            reader = csv.DictReader(csvfile)

            for row in reader:
                data.append(row)

        return data
```

ApolloAutomation/utils/excel_reader.py

```
import os
from openpyxl import load_workbook


class ExcelReader:

    @staticmethod
    def read_excel(file_name, sheet_name):

        data = []

        base_dir = os.path.dirname(os.path.dirname(__file__))

        file_path = os.path.join(
            base_dir,
            "data",
            file_name
        )

        workbook = load_workbook(file_path)

        sheet = workbook[sheet_name]

        headers = [cell.value for cell in sheet[1]]

        for row in sheet.iter_rows(min_row=2, values_only=True):

            row_data = dict(zip(headers, row))

            data.append(row_data)

        workbook.close()

        return data

    @staticmethod
    def read_shop_by_category_excel():
        return ExcelReader.read_excel(
            "test_data.xlsx",
            "ShopByCategory"
        )
```

ApolloAutomation/utils/logger.py

```
import logging
import os


class LogGen:

    @staticmethod
    def loggen():
```

```
        log_dir = "logs"

        if not os.path.exists(log_dir):
            os.makedirs(log_dir)

        logger = logging.getLogger()

        logger.setLevel(logging.INFO)

        if not logger.handlers:

            file_handler = logging.FileHandler(
                "logs/automation.log"
            )

            formatter = logging.Formatter(
                "%(asctime)s - %(levelname)s - %(message)s"
            )

            file_handler.setFormatter(formatter)

            logger.addHandler(file_handler)

            console_handler = logging.StreamHandler()

            console_handler.setFormatter(formatter)

            logger.addHandler(console_handler)

        return logger
```

ApolloAutomation/utils/screenshot_util.py

```
import os
from datetime import datetime

from utils.logger import LogGen

logger = LogGen.loggen()


class ScreenshotUtil:

    @staticmethod
    def capture_screenshot(driver, screenshot_name="screenshot"):

        try:
            logger.info("Starting screenshot capture process")

            base_dir = os.path.dirname(os.path.dirname(__file__))

            screenshot_dir = os.path.join(
                base_dir,
                "reports",
                "screenshots"
            )

            if not os.path.exists(screenshot_dir):
                os.makedirs(screenshot_dir)
                logger.info("Screenshots directory created successfully")

            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

            clean_name = (
                screenshot_name
                .replace(" ", "_")
                .replace("/", "_")
                .replace("\\", "_")
                .replace(":", "_")
                .replace("[", "_")
                .replace("]", "_")
                .lower()
            )

            screenshot_path = os.path.join(
                screenshot_dir,
                f"{clean_name}_{timestamp}.png"
            )

            driver.save_screenshot(screenshot_path)

            logger.info(f"Screenshot captured successfully: {screenshot_path}")

            print(f"Screenshot saved: {screenshot_path}")

            return screenshot_path

        except Exception as e:
            logger.error("Failed to capture screenshot")
            logger.error(f"Error: {str(e)}")
            raise
```

BBD_Apollo

BBD_Apollo/README.md

````
# Bbd_Project - Behave BDD Automation Framework

This project is created in BDD format using Selenium, Behave, Page Object Model, logging, screenshots, and Allure reporting.

## Folder Structure

```text
Bbd_Project
├── config
│   └── config.ini
├── data
│   ├── login_data.csv
│   └── shop_by_category_data.csv
├── features
│   ├── login.feature
│   ├── shop_by_category.feature
│   ├── environment.py
│   └── steps
│       ├── login_steps.py
│       └── shop_steps.py
├── locators
│   ├── login_locators.py
│   └── shop_locators.py
├── logs
├── pages
│   ├── base_page.py
│   ├── login_page.py
│   ├── signup_page.py
│   └── shop_by_category_page.py
├── reports
│   ├── allure-results
│   └── screenshots
├── utils
│   ├── config_reader.py
│   ├── csv_reader.py
│   ├── logger.py
│   ├── screenshot_util.py
│   └── waits.py
├── behave.ini
├── requirements.txt
└── runtests.py
````

BBD_Apollo/behave.ini

```
[behave]
paths = features
show_skipped = false
show_timings = true
format = pretty
stdout_capture = false
stderr_capture = false
log_capture = false
```

BBD_Apollo/config/config.ini

```
[app]
browser = chrome
base_url = https://www.apollo247.com/
implicit_wait = 10
timeout = 20
headless = false
mobile_number = 8757726775
```

BBD_Apollo/data/login_data.csv

```
mobile_number,expected_result
8757726775,success
```

BBD_Apollo/data/shop_by_category_data.csv

```
category_name,expected_result
Health Monitors,visible
Pain Relief,visible
Baby Care,visible
Fake Category,not_visible
Invalid Product,not_visible
```

BBD_Apollo/features/environment.py

```
import os
import time
import allure

from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.chrome.options import Options as ChromeOptions
from webdriver_manager.chrome import ChromeDriverManager

from utils.config_reader import ConfigReader
from utils.logger import LogGen
from utils.screenshot_util import ScreenshotUtil

logger = LogGen.loggen()
def before_all(context):
```
```

```
    logger.info("======================================")
    logger.info("BDD AUTOMATION EXECUTION STARTED")

    browser = ConfigReader.get_browser().strip().lower()
    headless = ConfigReader.is_headless()

    logger.info(f"Browser from config: {browser}")
    logger.info(f"Headless mode: {headless}")

    if browser == "chrome":

        chrome_options = ChromeOptions()

        chrome_options.add_argument("--start-maximized")
        chrome_options.add_argument("--disable-notifications")
        chrome_options.add_argument("--disable-infobars")
        chrome_options.add_argument("--disable-extensions")

        if headless:
            chrome_options.add_argument("--headless=new")

        context.driver = webdriver.Chrome(
            service=ChromeService(ChromeDriverManager().install()),
            options=chrome_options
        )

    else:

        raise Exception("Only Chrome browser is supported")

    context.driver.maximize_window()

    logger.info("Chrome browser launched successfully")


def before_scenario(context, scenario):

    logger.info("------------------------------------------------")
    logger.info(f"Scenario started: {scenario.name}")
    logger.info("------------------------------------------------")


def attach_log_to_allure(scenario_name):

    try:

        log_path = os.path.join(
            os.getcwd(),
            "logs",
            "automation.log"
        )

        if os.path.exists(log_path):

            allure.attach.file(
                log_path,
                name=f"LOG_{scenario_name}",
                attachment_type=allure.attachment_type.TEXT
            )

            logger.info("Log file attached to Allure report")

        else:

            logger.info("Log file not found, skipping Allure log attachment")

    except Exception as e:

        logger.info(f"Failed to attach log to Allure report: {e}")


def after_scenario(context, scenario):

    safe_name = (
        scenario.name
        .replace(" ", "_")
        .replace("/", "_")
        .replace("\\", "_")
        .replace(":", "_")
        .lower()
    )

    logger.info("------------------------------------------------")
    logger.info(f"Scenario finished: {scenario.name}")
    logger.info(f"Scenario status: {scenario.status.name}")
    logger.info("------------------------------------------------")

    if scenario.status.name == "passed":

        screenshot_path = ScreenshotUtil.capture_screenshot(
            context.driver,
            f"passed_{safe_name}"
        )

    else:

        screenshot_path = ScreenshotUtil.capture_screenshot(
            context.driver,
            f"failed_{safe_name}"
        )
    try:
```

```
        with open(screenshot_path, "rb") as image_file:

            allure.attach(
                image_file.read(),
                name=f"SCREENSHOT_{safe_name}",
                attachment_type=allure.attachment_type.PNG
            )

        logger.info("Screenshot attached to Allure report")

    except Exception as e:

        logger.info(f"Allure screenshot attachment skipped: {e}")

    attach_log_to_allure(safe_name)


def after_all(context):

    logger.info("Waiting 6 seconds before closing browser")

    time.sleep(6)

    context.driver.quit()

    logger.info("Browser closed successfully")
    logger.info("BDD AUTOMATION EXECUTION FINISHED")
    logger.info("============================================")
```

BBD_Apollo/features/login.feature

```
Feature: Apollo login

  Scenario: Login with valid mobile number and OTP
    Given user opens Apollo application
    When user enters valid mobile number
    Then OTP screen should be visible
    When user enters OTP manually
    Then login should be submitted successfully
```

BBD_Apollo/features/shop_by_category.feature

```
Feature: Apollo shop by category

  Scenario Outline: Validate category visibility
    Given user opens Apollo application
    Then category "<category_name>" should be "<expected_result>"

    Examples:
      | category_name    | expected_result |
      | Health Monitors  | visible         |
      | Pain Relief      | visible         |
      | Baby Care        | visible         |
      | Fake Category    | not_visible     |
      | Invalid Product  | not_visible     |

  Scenario: Open Health Monitors category
    Given user opens Apollo application
    When user opens Health Monitors category
    Then Health Monitors page should be opened

  Scenario: Apply Doctor S Choice filter
    Given user opens Apollo application
    When user opens Health Monitors category
    Then Health Monitors page should be opened
    Then Brands filter should be visible
    When user applies Doctor S Choice filter
    Then Health Monitors page should be opened

  Scenario: Add Doctor S Choice product to cart
    Given user opens Apollo application
    When user adds Doctor S Choice product to cart
    Then cart page should be opened
    Then selected product should be present in cart

  Scenario: Proceed after adding product to cart
    Given user opens Apollo application
    When user adds Doctor S Choice product to cart
    Then cart page should be opened
    Then selected product should be present in cart
    When user clicks Proceed button
    Then next cart step should be opened
```

BBD_Apollo/features/steps/login_steps.py

```
from behave import given, when, then

from pages.login_page import LoginPage
from utils.config_reader import ConfigReader


@given("user opens Apollo application")
def step_open_application(context):

    context.driver.get(ConfigReader.get_base_url())


@when("user enters valid mobile number")
def step_enter_mobile_number(context):
    login_page = LoginPage(context.driver)
```

```
    context.login_page = login_page

    login_page.login_with_mobile_number(
        ConfigReader.get("mobile_number")
    )


@then("OTP screen should be visible")
def step_otp_screen_visible(context):

    assert context.login_page.is_otp_screen_visible(), \
        "OTP screen should be visible after entering mobile number"


@when("user enters OTP manually")
def step_enter_otp_manually(context):

    print("\nEnter OTP manually in browser.")
    print("Do not press anything in terminal.")
    print("Automation will continue automatically after OTP is entered.")
    print("Maximum wait time is 60 seconds.\n")

    context.otp_submitted = (
        context.login_page.wait_for_otp_entry_and_submit(
            timeout=60
        )
    )


@then("login should be submitted successfully")
def step_login_submitted(context):

    assert context.otp_submitted is True, \
        "OTP was not entered or submitted successfully"
```

BBD_Apollo/features/steps/shop_steps.py

```
from behave import when, then

from pages.shop_by_category_page import ShopByCategoryPage


@then('category "{category_name}" should be "{expected_result}"')
def step_validate_category(context, category_name, expected_result):

    shop_page = ShopByCategoryPage(context.driver)

    if expected_result == "visible":

        actual_result = shop_page.is_category_visible(
            category_name,
            timeout=5
        )

        assert actual_result is True, \
            f"{category_name} category should be visible"

    elif expected_result == "not_visible":

        actual_result = shop_page.is_category_visible(
            category_name,
            timeout=2
        )

        assert actual_result is False, \
            f"{category_name} category should not be visible"


@when("user opens Health Monitors category")
def step_open_health_monitors(context):

    shop_page = ShopByCategoryPage(context.driver)

    context.shop_page = shop_page

    shop_page.click_health_monitors()


@then("Health Monitors page should be opened")
def step_health_monitors_opened(context):

    shop_page = getattr(
        context,
        "shop_page",
        ShopByCategoryPage(context.driver)
    )

    assert shop_page.is_health_monitors_page_opened(), \
        "Health Monitors page did not open"


@then("Brands filter should be visible")
def step_brands_filter_visible(context):

    assert context.shop_page.is_brands_filter_visible(), \
        "Brands filter is not visible"


@when("user applies Doctor S Choice filter")
def step_apply_doctor_filter(context):
    context.shop_page.apply_doctor_s_choice_filter()
```

```
@when("user adds Doctor S Choice product to cart")
def step_add_product_to_cart(context):

    shop_page = ShopByCategoryPage(context.driver)

    context.shop_page = shop_page

    context.product_name = (
        shop_page.add_exact_product_to_cart_with_retry(
            max_attempts=3
        )
    )
    assert context.product_name is not None, \
        "Product name was not captured"


@then("cart page should be opened")
def step_cart_page_opened(context):

    assert context.shop_page.is_cart_page_opened(), \
        "Cart page did not open"


@then("selected product should be present in cart")
def step_product_present_in_cart(context):

    assert context.shop_page.is_exact_product_present_in_cart(
        context.product_name
    ), f"Expected product not found in cart: {context.product_name}"


@when("user clicks Proceed button")
def step_click_proceed(context):

    context.shop_page.click_proceed_button()


@then("next cart step should be opened")
def step_next_cart_step_opened(context):

    assert context.shop_page.is_after_proceed_page_opened(), \
        "Page did not move forward after clicking Proceed"
```

BBD_Apollo/locators/login_locators.py

```
from selenium.webdriver.common.by import By


class LoginLocators:

    LOGIN_ICON = (
        By.XPATH,
        "//*[contains(text(),'Login') or contains(text(),'Sign in') or contains(text(),'Sign In')]"
    )

    MOBILE_INPUT = (
        By.XPATH,
        "//input"
    )

    CONTINUE_BUTTON = (
        By.XPATH,
        "//button[contains(text(),'Continue')]"
    )

    OTP_SCREEN_TEXT = (
        By.XPATH,
        "//*[contains(text(),'OTP') or contains(text(),'otp') or contains(text(),'Resend') or contains(text(),'sent')]"
    )

    VERIFY_BUTTON = (
        By.XPATH,
        "//button[contains(text(),'Verify') or contains(text(),'Continue') or contains(text(),'Submit')]"
    )
```

BBD_Apollo/locators/shop_locators.py

```
from selenium.webdriver.common.by import By


class ShopLocators:

    HEALTH_MONITORS_TEXT = (
        By.XPATH,
        "//*[normalize-space()='Health Monitors']"
    )

    EXACT_ADD_BUTTON = (
        By.XPATH,
        "(//button[@aria-label='Add' and .//span[normalize-space()='Add']])[1]"
    )

    CART_ICON = (
        By.XPATH,
        "//*[contains(@href,'cart') or contains(@class,'cart') or contains(text(),'Cart')]"
    )

    CART_TEXT = (
        By.XPATH,
        "//*[contains(text(),'Cart') or contains(text(),'cart')]"
```

```
    )

    EMPTY_CART_TEXT = (
        By.XPATH,
        "//*[contains(text(),'YOUR CART IS EMPTY') or contains(text(),'Your cart is empty') or contains(text(),'empty') or
contains(text(),'Empty')]"
    )
```

BBD_Apollo/pages/__init__.py

```
[empty file]
```

BBD_Apollo/pages/base_page.py

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

from utils.logger import LogGen

logger = LogGen.loggen()


class BasePage:

    def __init__(self, driver, timeout=20):

        self.driver = driver
        self.wait = WebDriverWait(driver, timeout)

    def get_element(
        self,
        locator,
        condition=EC.visibility_of_element_located,
        timeout=20
    ):

        logger.info(f"Locating element: {locator}")

        wait = WebDriverWait(self.driver, timeout)

        return wait.until(condition(locator))

    def click(self, locator, timeout=10):

        logger.info(f"Clicking element: {locator}")

        self.get_element(
            locator,
            EC.element_to_be_clickable,
            timeout
        ).click()

    def type_text(self, locator, text, timeout=10):

        logger.info(f"Typing text into element: {locator}")

        element = self.get_element(
            locator,
            EC.visibility_of_element_located,
            timeout
        )

        element.clear()
        element.send_keys(text)

    def is_visible(self, locator, timeout=5):

        try:
            return self.get_element(
                locator,
                EC.visibility_of_element_located,
                timeout
            ).is_displayed()

        except Exception:
            return False

    def scroll_to_element(self, locator, timeout=10):

        element = self.get_element(
            locator,
            EC.visibility_of_element_located,
            timeout
        )

        self.driver.execute_script(
            "arguments[0].scrollIntoView({block:'center'});",
            element
        )

    def js_click(self, locator, timeout=10):

        element = self.get_element(
            locator,

            EC.visibility_of_element_located,
            timeout
        )

        self.driver.execute_script(
            "arguments[0].click();",
```

```
            element
        )

    def close_popup_if_present(self):

        logger.info("Checking popup on page")

        self.driver.execute_script(
            """
            const popup = document.querySelector('ct-web-popup-imageonly');

            if (popup) {
                popup.remove();
            }
            """
        )
```

BBD_Apollo/pages/login_page.py

```
from selenium.webdriver.common.keys import Keys
from selenium.common.exceptions import StaleElementReferenceException
import time

from pages.base_page import BasePage
from locators.login_locators import LoginLocators
from utils.logger import LogGen

logger = LogGen.loggen()


class LoginPage(BasePage):

    def open_login_popup(self):

        logger.info("Opening login popup")

        self.close_popup_if_present()

        time.sleep(2)

        try:
            self.click(LoginLocators.LOGIN_ICON, timeout=8)

            time.sleep(2)

            return

        except Exception as e:
            logger.info(f"Normal login click failed: {e}")

        try:
            self.js_click(LoginLocators.LOGIN_ICON, timeout=8)

            time.sleep(2)

            return

        except Exception as e:
            logger.info(f"JavaScript login click failed: {e}")

        clicked = self.driver.execute_script(
            """
            const elements = document.querySelectorAll(
                'button, div, span, p, a'
            );

            for (const element of elements) {

                const text = (
                    element.innerText ||
                    element.textContent ||
                    ''
                ).trim().toLowerCase();

                if (text.includes('login') || text.includes('sign in')) {
                    element.click();
                    return true;
                }
            }

            return false;
            """
        )

        if not clicked:
            raise Exception("Login button not found")

        time.sleep(2)

    def login_with_mobile_number(self, mobile_number):

        logger.info("Starting login with mobile number")

        self.open_login_popup()

        mobile_input = self.get_element(
            LoginLocators.MOBILE_INPUT,
            timeout=10
        )

        mobile_input.clear()
```

```
        mobile_input.send_keys(mobile_number)

        time.sleep(1)

        try:
            self.click(LoginLocators.CONTINUE_BUTTON, timeout=8)

        except Exception:
            mobile_input.send_keys(Keys.ENTER)

        time.sleep(3)

    def is_otp_screen_visible(self):

        logger.info("Checking OTP screen")

        return self.is_visible(
            LoginLocators.OTP_SCREEN_TEXT,
            timeout=10
        )

    def wait_for_otp_entry_and_submit(self, timeout=60):

        logger.info("Waiting for OTP entry")

        end_time = time.time() + timeout

        while time.time() < end_time:

            try:
                inputs = self.driver.find_elements(
                    *LoginLocators.MOBILE_INPUT
                )

                values = [
                    field.get_attribute("value") or ""
                    for field in inputs
                ]

                if any(len(value.strip()) >= 4 for value in values):

                    logger.info("OTP entered manually")

                    try:
                        inputs[-1].send_keys(Keys.ENTER)
                        logger.info("OTP submitted using Enter key")

                    except StaleElementReferenceException:
                        pass

                    time.sleep(2)

                    return True

            except Exception as e:
                logger.info(f"Waiting for OTP: {e}")

            time.sleep(1)

        return False
```

BBD_Apollo/pages/shop_by_category_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.common.action_chains import ActionChains
import time

from pages.base_page import BasePage
from locators.shop_locators import ShopLocators
from utils.logger import LogGen

logger = LogGen.loggen()


class ShopByCategoryPage(BasePage):

    HEALTH_MONITORS_URL = (
        "https://www.apollopharmacy.in/shop-by-category/apollo-brand-health-monitors"
    )

    def is_category_visible(self, category_name, timeout=5):

        self.close_popup_if_present()

        locator = (
            By.XPATH,
            f"//*[normalize-space()='{category_name}']"
        )

        return self.is_visible(locator, timeout=timeout)

    def click_health_monitors(self):

        logger.info("Opening Health Monitors category")

        self.close_popup_if_present()

        try:
            self.scroll_to_element(
                ShopLocators.HEALTH_MONITORS_TEXT,
```

```
            timeout=5
        )

        time.sleep(1)

        health_monitor_element = self.get_element(
            ShopLocators.HEALTH_MONITORS_TEXT,
            timeout=5
        )

        self.driver.execute_script(
            "arguments[0].click();",
            health_monitor_element
        )

        WebDriverWait(self.driver, 5).until(
            lambda driver:
            "apollo-brand-health-monitors" in driver.current_url
        )

    except Exception as e:

        logger.info(f"Health Monitors click failed: {e}")
        logger.info("Opening Health Monitors page directly")

        self.driver.get(self.HEALTH_MONITORS_URL)

        WebDriverWait(self.driver, 10).until(
            lambda driver:
            "apollo-brand-health-monitors" in driver.current_url
        )

    time.sleep(6)

def is_health_monitors_page_opened(self):

    current_url = self.driver.current_url

    logger.info(f"Current URL is: {current_url}")

    return "apollo-brand-health-monitors" in current_url
def is_brands_filter_visible(self):

    logger.info("Checking Brands filter visibility")

    time.sleep(5)

    page_text = self.driver.find_element(
        By.TAG_NAME,
        "body"
    ).text.lower()

    return "brands" in page_text

def open_brands_filter(self):

    logger.info("Opening Brands filter")

    self.close_popup_if_present()

    time.sleep(3)

    opened = self.driver.execute_script(
        """
        const elements = Array.from(
            document.querySelectorAll('label, div, span, h3, button')
        );

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = (
                element.innerText ||
                element.textContent ||
                ''
            ).trim().toLowerCase();

            if (text === 'brands' || text.includes('brands')) {

                element.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                const clickable =
```

```
                    element.closest('label, button, div') || element;

                clickable.click();

                return true;
            }
        }

        return false;
        """
    )

    assert opened is True, \
        "Brands filter was not found or not clicked"

    time.sleep(3)

def apply_doctor_s_choice_filter(self):

    logger.info("Applying Doctor S Choice brand filter")
    self.close_popup_if_present()

    self.open_brands_filter()

    time.sleep(3)

    clicked = self.driver.execute_script(
        """
        function normalizeText(value) {
            return (value || '')
                .replace(/['']/g, '')
                .replace(/[^a-zA-Z0-9]/g, ' ')
                .replace(/\\s+/g, ' ')
                .trim()
                .toLowerCase();
        }

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        window.scrollBy(0, 300);

        const elements = Array.from(
            document.querySelectorAll('label, div, span, p, h3, button')
        );

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = normalizeText(
                element.innerText || element.textContent
            );

            const isDoctorChoice =
                text.includes('doctor s choice') ||
                text.includes('doctors choice') ||
                (
                    text.includes('doctor') &&
                    text.includes('choice')
                );

            if (isDoctorChoice) {

                element.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                const clickable =
                    element.closest('label, button, div') || element;

                clickable.dispatchEvent(
                    new MouseEvent('mouseover', {bubbles: true})
                );

                clickable.dispatchEvent(
                    new MouseEvent('mousedown', {bubbles: true})
                );

                clickable.dispatchEvent(
                    new MouseEvent('mouseup', {bubbles: true})
                );

                clickable.dispatchEvent(
                    new MouseEvent('click', {bubbles: true})
                );

                return true;
```

```
            }
        }

        return false;
        """
    )

    assert clicked is True, \
        "Doctor S Choice filter option was not found or not clicked"

    time.sleep(5)

def get_exact_add_button(self):

    add_button = self.get_element(
        ShopLocators.EXACT_ADD_BUTTON,
        timeout=20
    )

    self.driver.execute_script(
        "arguments[0].scrollIntoView({block:'center'});",
        add_button
    )

    time.sleep(2)

    return add_button

def get_product_name_from_add_button(self, add_button):

    product_name = self.driver.execute_script(
        """
        const addBtn = arguments[0];

        let parent = addBtn;

        for (let i = 0; i < 12; i++) {

            if (!parent) {
                break;
            }

            const text = (
                parent.innerText ||
                parent.textContent ||
                ''
            ).trim();

            if (
                text.length > 10 &&
                text.toLowerCase().includes('add')
            ) {

                const lines = text
                    .split('\\n')
                    .map(line => line.trim())
                    .filter(line => line.length > 0);

                for (const line of lines) {

                    const lower = line.toLowerCase();

                    if (
                        !lower.includes('add') &&
                        !lower.includes('rs') &&
                        !lower.includes('mrp') &&
                        !lower.includes('off') &&
                        !lower.includes('%') &&
                        !lower.includes('rating') &&
                        !lower.includes('cart') &&
                        !lower.includes('bestseller') &&
                        line.length > 5
                    ) {
                        return line;
                    }
                }
            }

            parent = parent.parentElement;
        }
        return '';
        """,
        add_button
    ).strip()

    if product_name == "":
        raise Exception("Product name could not be captured")

    return product_name

def click_exact_add_button_once(self, add_button):

    self.driver.execute_script(
        "arguments[0].scrollIntoView({block:'center'});",
        add_button
    )

    time.sleep(1)

    try:
```

```
            add_button.click()
            return

        except Exception:
            pass

        try:
            ActionChains(self.driver)\
                .move_to_element(add_button)\
                .pause(1)\
                .click(add_button)\
                .perform()

            return

        except Exception:
            pass

        self.driver.execute_script(
            """
            const button = arguments[0];

            button.scrollIntoView({block:'center'});

            button.dispatchEvent(
                new MouseEvent('mouseover', {bubbles: true})
            );

            button.dispatchEvent(
                new MouseEvent('mousedown', {bubbles: true})
            );

            button.dispatchEvent(
                new MouseEvent('mouseup', {bubbles: true})
            );

            button.dispatchEvent(
                new MouseEvent('click', {bubbles: true})
            );
            """,
            add_button
        )

    def go_to_cart(self):

        logger.info("Opening cart")

        self.close_popup_if_present()

        try:
            self.click(ShopLocators.CART_ICON, timeout=10)

        except Exception:
            self.driver.get("https://www.apollopharmacy.in/cart")

        WebDriverWait(self.driver, 15).until(
            lambda driver:
            "cart" in driver.current_url.lower()
            or len(driver.find_elements(*ShopLocators.CART_TEXT)) > 0
        )

        time.sleep(5)

    def is_cart_page_opened(self):

        return (
            "cart" in self.driver.current_url.lower()
            or self.is_visible(ShopLocators.CART_TEXT, timeout=5)
        )

    def is_cart_empty(self):

        empty_items = self.driver.find_elements(
            *ShopLocators.EMPTY_CART_TEXT
        )

        for item in empty_items:

            try:
                if item.is_displayed():
                    return True

            except Exception:
                continue

        return False

    def is_exact_product_present_in_cart(self, product_name):

        time.sleep(3)

        if self.is_cart_empty():
            return False

        cart_text = self.driver.find_element(
            By.TAG_NAME,
            "body"
        ).text

        return product_name.lower() in cart_text.lower()
```

```
def add_exact_product_to_cart_with_retry(self, max_attempts=3):

    product_name = None

    for attempt in range(1, max_attempts + 1):

        logger.info(f"Add product attempt: {attempt}")

        self.driver.get(self.HEALTH_MONITORS_URL)

        WebDriverWait(self.driver, 10).until(
            lambda driver:
            "apollo-brand-health-monitors" in driver.current_url
        )

        time.sleep(6)

        self.apply_doctor_s_choice_filter()

        add_button = self.get_exact_add_button()

        product_name = self.get_product_name_from_add_button(
            add_button
        )

        self.click_exact_add_button_once(add_button)

        time.sleep(7)

        self.go_to_cart()

        if (
            self.is_cart_page_opened()
            and self.is_exact_product_present_in_cart(product_name)
        ):
            return product_name

    raise AssertionError(
        f"Product was not added to cart after {max_attempts} attempts"
    )

def click_proceed_button(self):

    assert self.is_cart_page_opened(), \
        "Cart page is not opened, cannot click Proceed"

    assert not self.is_cart_empty(), \
        "Cart is empty, cannot click Proceed"

    self.driver.execute_script(
        "window.scrollTo(0, document.body.scrollHeight);"
    )

    time.sleep(3)

    clicked = self.driver.execute_script(
        """
        const elements = Array.from(
            document.querySelectorAll('button, span, div')
        );

        function normalizeText(value) {
            return (value || '')
                .replace(/\\s+/g, ' ')
                .trim()
                .toLowerCase();
        }

        function isVisible(element) {
            const style = window.getComputedStyle(element);
            const rect = element.getBoundingClientRect();

            return (
                style.display !== 'none' &&
                style.visibility !== 'hidden' &&
                rect.width > 0 &&
                rect.height > 0
            );
        }

        for (const element of elements) {

            if (!isVisible(element)) {
                continue;
            }

            const text = normalizeText(
                element.innerText || element.textContent
            );

            const title = normalizeText(
                element.getAttribute('title')
            );

            if (text === 'proceed' || title === 'proceed') {

                let clickable = element.closest('button');

                if (!clickable) {
                    clickable = element;
```

```
                }

                clickable.scrollIntoView({
                    block: 'center',
                    inline: 'center'
                });

                clickable.dispatchEvent(
                    new MouseEvent('mouseover', {bubbles: true})
                );

                clickable.dispatchEvent(
                    new MouseEvent('mousedown', {bubbles: true})
                );

                clickable.dispatchEvent(
                    new MouseEvent('mouseup', {bubbles: true})
                );
                clickable.dispatchEvent(
                    new MouseEvent('click', {bubbles: true})
                );

                return true;
            }
        }

            return false;
            """
        )

        assert clicked is True, \
            "Proceed button was not found or not clicked"

        time.sleep(5)

    def is_after_proceed_page_opened(self):

        time.sleep(5)

        page_text = self.driver.find_element(
            By.TAG_NAME,
            "body"
        ).text.lower()

        current_url = self.driver.current_url.lower()

        return (
            "medicines-cart" in current_url
            or "select address" in page_text
            or "delivery address" in page_text
            or "add address" in page_text
            or "address" in page_text
            or "payment" in page_text
            or "login" in page_text
            or "continue" in page_text
            or "checkout" in current_url
            or "address" in current_url
            or "payment" in current_url
        )
```

BBD_Apollo/pretty.output

```
Feature: Apollo login # features/login.feature:1

  Scenario: Login with valid mobile number and OTP  # features/login.feature:3
    Given user opens Apollo application             # features/steps/login_steps.py:7
    When user enters valid mobile number            # features/steps/login_steps.py:13
    Then OTP screen should be visible               # features/steps/login_steps.py:25
    When user enters OTP manually                   # features/steps/login_steps.py:32
    Then login should be submitted successfully     # features/steps/login_steps.py:47

Feature: Apollo shop by category # features/shop_by_category.feature:1

  Scenario Outline: Validate category visibility -- @1.1   # features/shop_by_category.feature:9
    Given user opens Apollo application                    # features/steps/login_steps.py:7
    Then category "Health Monitors" should be "visible"    # features/steps/shop_steps.py:6

  Scenario Outline: Validate category visibility -- @1.2   # features/shop_by_category.feature:10
    Given user opens Apollo application                    # features/steps/login_steps.py:7
    Then category "Pain Relief" should be "visible"        # features/steps/shop_steps.py:6

  Scenario Outline: Validate category visibility -- @1.3   # features/shop_by_category.feature:11
    Given user opens Apollo application                    # features/steps/login_steps.py:7
    Then category "Baby Care" should be "visible"          # features/steps/shop_steps.py:6

  Scenario Outline: Validate category visibility -- @1.4   # features/shop_by_category.feature:12
    Given user opens Apollo application                    # features/steps/login_steps.py:7
    Then category "Fake Category" should be "not_visible"  # features/steps/shop_steps.py:6

  Scenario Outline: Validate category visibility -- @1.5    # features/shop_by_category.feature:13
    Given user opens Apollo application                     # features/steps/login_steps.py:7
    Then category "Invalid Product" should be "not_visible" # features/steps/shop_steps.py:6

  Scenario: Open Health Monitors category      # features/shop_by_category.feature:15
    Given user opens Apollo application        # features/steps/login_steps.py:7
    When user opens Health Monitors category   # features/steps/shop_steps.py:32
    Then Health Monitors page should be opened # features/steps/shop_steps.py:42

  Scenario: Apply Doctor S Choice filter       # features/shop_by_category.feature:20
    Given user opens Apollo application        # features/steps/login_steps.py:7
```

```
    When user opens Health Monitors category   # features/steps/shop_steps.py:32
    Then Health Monitors page should be opened # features/steps/shop_steps.py:42
    Then Brands filter should be visible       # features/steps/shop_steps.py:55
    When user applies Doctor S Choice filter   # features/steps/shop_steps.py:62
    Then Health Monitors page should be opened # features/steps/shop_steps.py:42

  Scenario: Add Doctor S Choice product to cart     # features/shop_by_category.feature:28
    Given user opens Apollo application             # features/steps/login_steps.py:7
    When user adds Doctor S Choice product to cart  # features/steps/shop_steps.py:68
    Then cart page should be opened                 # features/steps/shop_steps.py:85
    Then selected product should be present in cart # features/steps/shop_steps.py:92

  Scenario: Proceed after adding product to cart    # features/shop_by_category.feature:34
    Given user opens Apollo application             # features/steps/login_steps.py:7
    When user adds Doctor S Choice product to cart  # features/steps/shop_steps.py:68
    Then cart page should be opened                 # features/steps/shop_steps.py:85
    Then selected product should be present in cart # features/steps/shop_steps.py:92
    When user clicks Proceed button                 # features/steps/shop_steps.py:100
    Then next cart step should be opened            # features/steps/shop_steps.py:106
```

BBD_Apollo/requirements.txt

```
selenium
webdriver-manager
behave
allure-behave
allure-python-commons
configparser
```

BBD_Apollo/runtests.py

```
import os
import shutil
import subprocess
from datetime import datetime

from utils.logger import LogGen

logger = LogGen.loggen()


def clean_folder(folder_path, folder_name):

    if os.path.exists(folder_path):

        logger.info(f"Deleting old {folder_name} folder")

        shutil.rmtree(folder_path)

    os.makedirs(folder_path, exist_ok=True)


def run_command(command):

    logger.info(f"Running command: {' '.join(command)}")

    result = subprocess.run(
        command,
        shell=True,
        text=True
    )

    return result.returncode


def main():

    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

    logger.info("============================================")
    logger.info("AUTOMATION EXECUTION STARTED")
    logger.info(f"Execution timestamp: {timestamp}")

    allure_results = os.path.join(
        "reports",
        "allure-results"
    )

    allure_report = os.path.join(
        "reports",
        "allure-report"
    )

    clean_folder(
        allure_results,
        "allure-results"
    )

    if os.path.exists(allure_report):

        logger.info("Deleting old allure-report folder")

        shutil.rmtree(allure_report)

    behave_command = [
        "behave",
        "-f",
        "allure_behave.formatter:AllureFormatter",
        "-o",
        allure_results,
```

```
        "features"
    ]

    behave_status = run_command(behave_command)

    generate_command = [
        "allure",
        "generate",
        allure_results,
        "-o",
        allure_report,
        "--clean"
    ]

    report_status = run_command(generate_command)

    if behave_status == 0 and report_status == 0:

        logger.info("AUTOMATION EXECUTION COMPLETED SUCCESSFULLY")

        print("\n==========================================")
        print("BDD automation completed successfully")
        print(f"Allure report generated at: {allure_report}")
        print("Open report using: allure open reports/allure-report")
        print("==========================================\n")

    else:

        logger.error("AUTOMATION EXECUTION COMPLETED WITH FAILURES")

        print("\n==========================================")
        print("BDD automation completed with failures")
        print("Check logs and reports/allure-results")
        print("==========================================\n")

    logger.info("==========================================")


if __name__ == "__main__":
    main()
```

BBD_Apollo/utils/__init__.py

```
[empty file]
```

BBD_Apollo/utils/config_reader.py

```
import configparser
import os


class ConfigReader:

    _config = None

    @classmethod
    def load_config(cls):

        if cls._config is not None:
            return cls._config

        config_path = os.path.join(
            os.path.dirname(os.path.dirname(__file__)),
            "config",
            "config.ini"
        )

        if not os.path.exists(config_path):
            raise FileNotFoundError(
                f"Config file not found at path: {config_path}"
            )

        parser = configparser.ConfigParser()
        parser.read(config_path, encoding="utf-8")

        if "app" not in parser:
            raise KeyError("[app] section is missing in config.ini")

        cls._config = parser

        return cls._config

    @classmethod
    def get(cls, key):

        config = cls.load_config()

        if key not in config["app"]:
            raise KeyError(f"{key} not found in config.ini")

        return config["app"][key].strip()

    @classmethod
    def get_base_url(cls):
        return cls.get("base_url")

    @classmethod
    def get_browser(cls):
        return cls.get("browser")
    @classmethod
```

```
    def get_timeout(cls):
        return int(cls.get("timeout"))

    @classmethod
    def get_implicit_wait(cls):
        return int(cls.get("implicit_wait"))

    @classmethod
    def get_mobile_number(cls):
        return cls.get("mobile_number")

    @classmethod
    def is_headless(cls):
        return cls.get("headless").lower() == "true"
```

BBD_Apollo/utils/csv_reader.py

```
import csv
import os


class CSVReader:

    @staticmethod
    def read_csv(file_name):

        file_path = os.path.join(
            os.getcwd(),
            "data",
            file_name
        )

        data = []

        with open(file_path, mode="r", encoding="utf-8-sig") as file:

            reader = csv.DictReader(file)

            for row in reader:
                data.append(row)

        return data
```

BBD_Apollo/utils/logger.py

```
import logging
import os


class LogGen:

    @staticmethod
    def loggen():

        log_dir = "logs"

        if not os.path.exists(log_dir):
            os.makedirs(log_dir)

        logger = logging.getLogger()
        logger.setLevel(logging.INFO)

        if not logger.handlers:

            file_handler = logging.FileHandler("logs/automation.log")

            formatter = logging.Formatter(
                "%(asctime)s - %(levelname)s - %(message)s"
            )

            file_handler.setFormatter(formatter)
            logger.addHandler(file_handler)

            console_handler = logging.StreamHandler()
            console_handler.setFormatter(formatter)
            logger.addHandler(console_handler)

        return logger
```

BBD_Apollo/utils/screenshot_util.py

```
import os
from datetime import datetime

from utils.logger import LogGen

logger = LogGen.loggen()


class ScreenshotUtil:

    @staticmethod
    def capture_screenshot(driver, screenshot_name="screenshot"):

        try:
            base_dir = os.path.dirname(os.path.dirname(__file__))

            screenshot_dir = os.path.join(
                base_dir,
                "reports",
```

```
            "screenshots"
        )

        if not os.path.exists(screenshot_dir):
            os.makedirs(screenshot_dir)

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

        clean_name = (
            screenshot_name
            .replace(" ", "_")
            .replace("/", "_")
            .replace("\\", "_")
            .replace(":", "_")
            .replace("[", "_")
            .replace("]", "_")
            .lower()
        )

        screenshot_path = os.path.join(
            screenshot_dir,
            f"{clean_name}_{timestamp}.png"
        )

        driver.save_screenshot(screenshot_path)

        logger.info(
            f"Screenshot captured successfully: {screenshot_path}"
        )

        print(f"Screenshot saved: {screenshot_path}")

        return screenshot_path

    except Exception as e:

        logger.error("Failed to capture screenshot")
        logger.error(str(e))

        raise
```

BBD_Apollo/utils/waits.py

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC


class Waits:

    @staticmethod
    def visible(driver, locator, timeout=20):

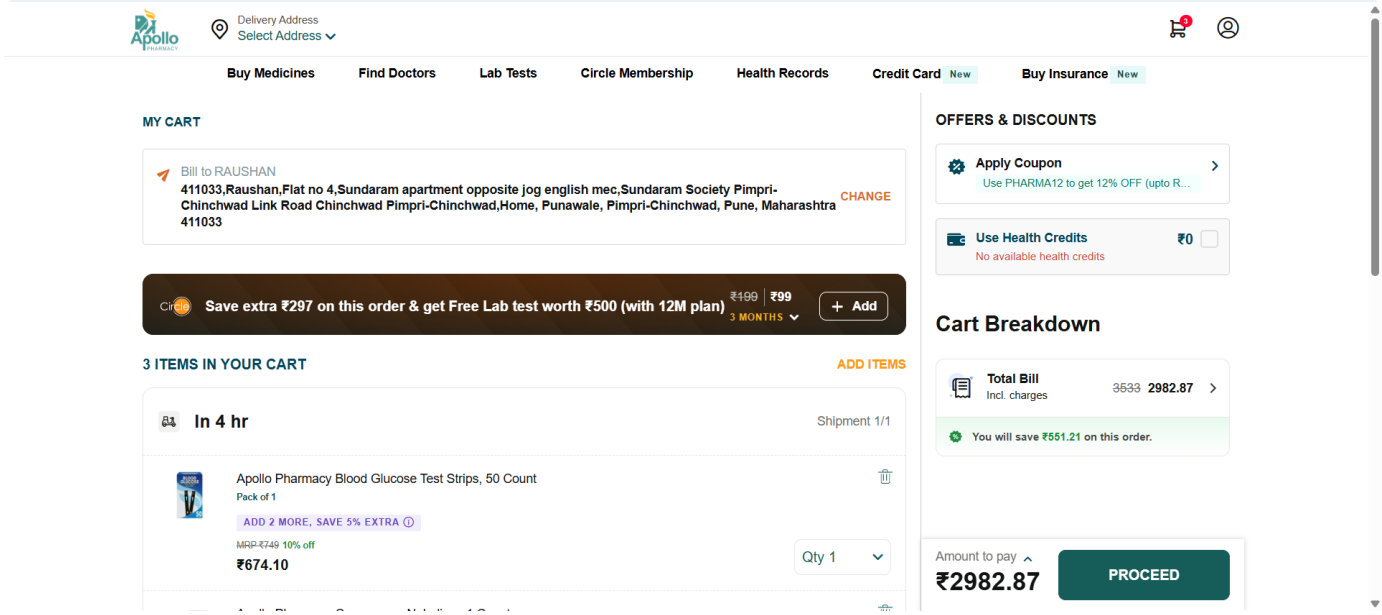
        return WebDriverWait(driver, timeout).until(
            EC.visibility_of_element_located(locator)
        )

    @staticmethod
    def clickable(driver, locator, timeout=20):

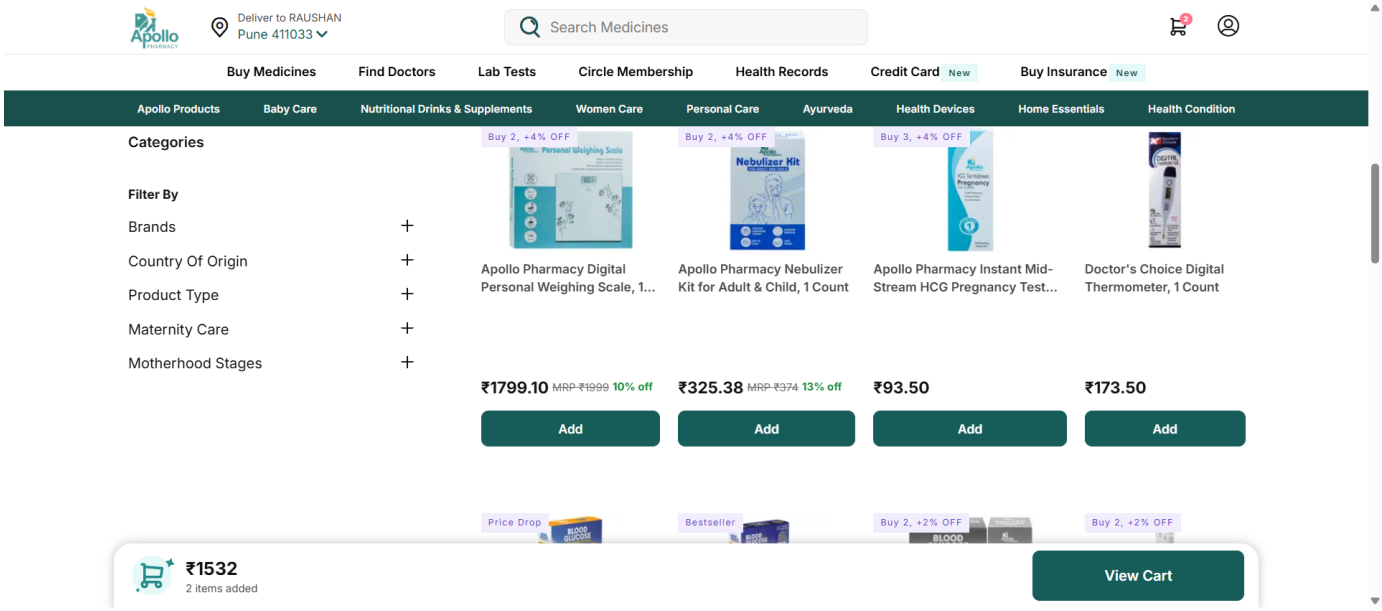
        return WebDriverWait(driver, timeout).until(
            EC.element_to_be_clickable(locator)
        )
```

5. Execution Screenshots

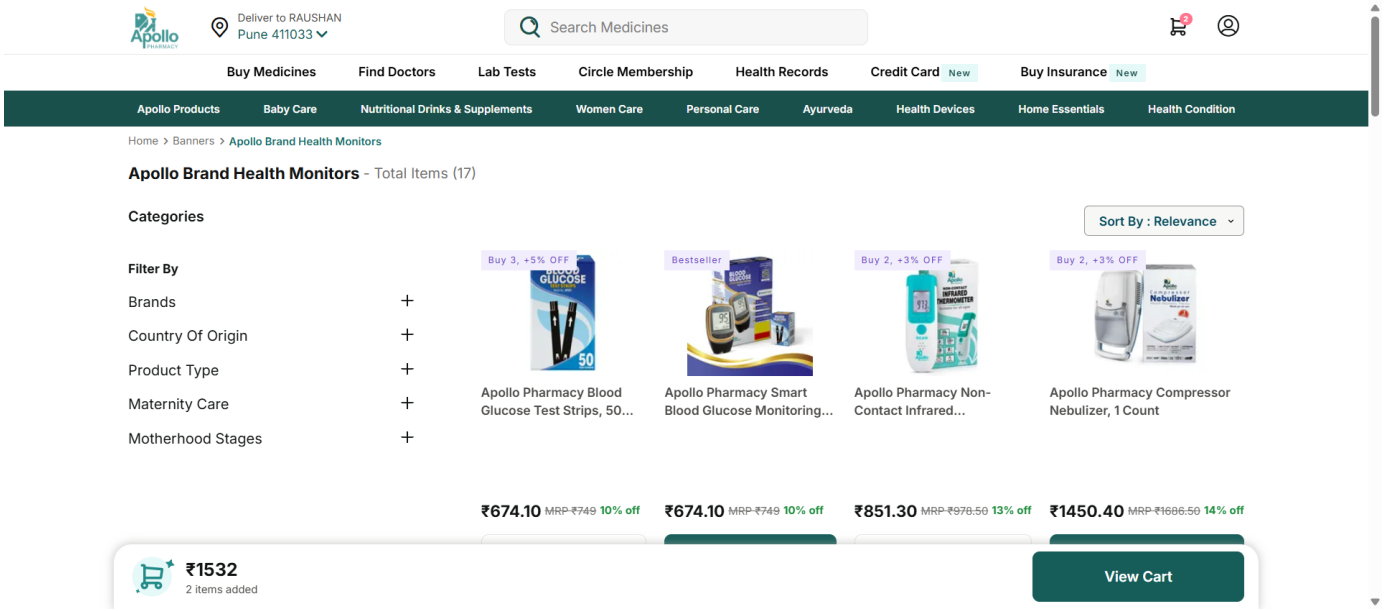
ApolloAutomation/reports/screenshots/passed_test_add_doctor_s_choice_product_to_cart_20260522_164306.png



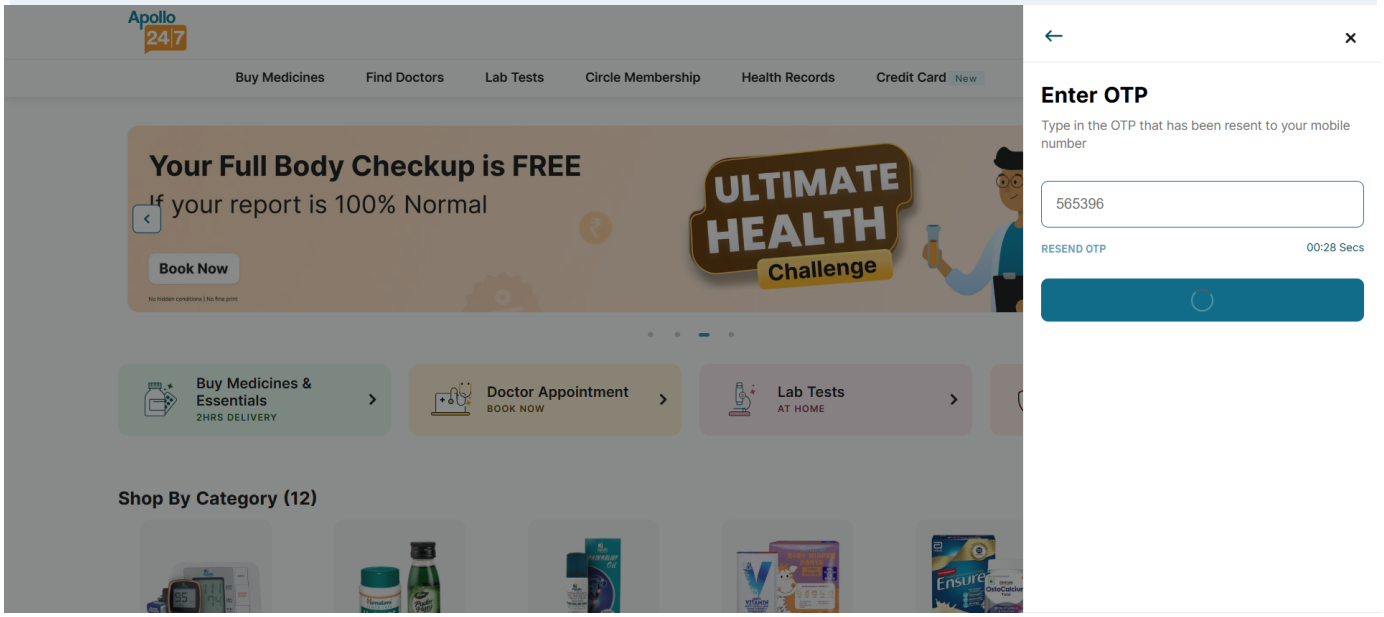
ApolloAutomation/reports/screenshots/passed_test_apply_doctor_s_choice_filter_after_health_monitors_20260522_164216.png



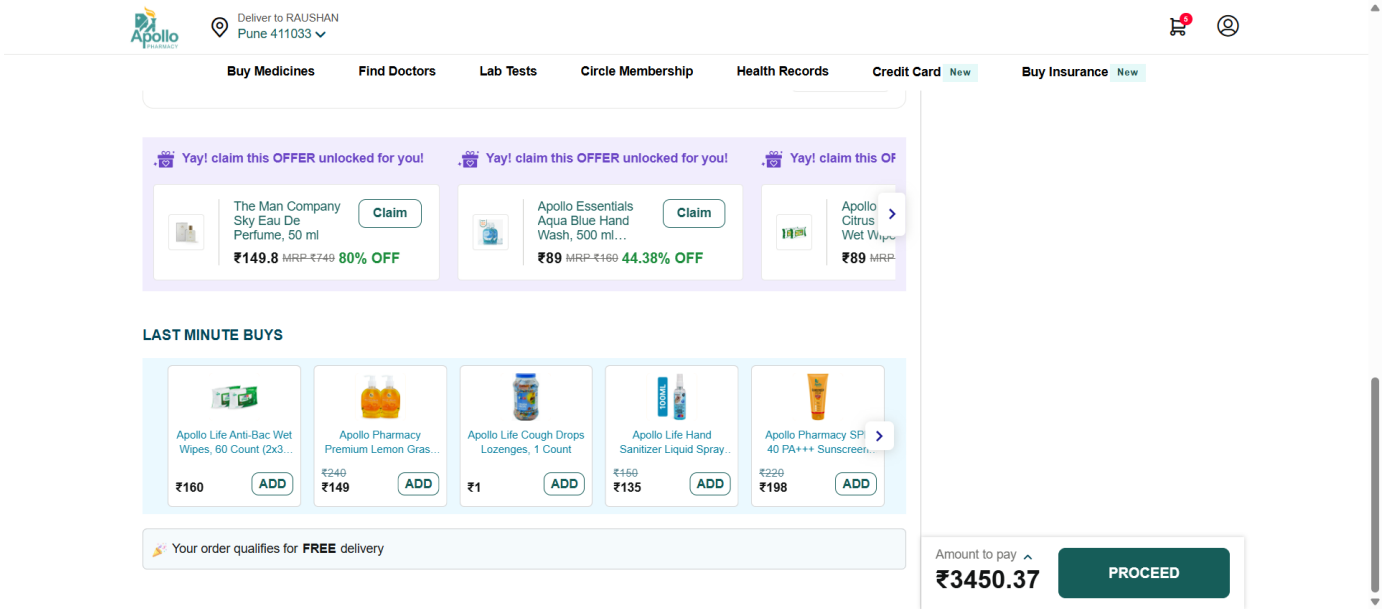
ApolloAutomation/reports/screenshots/passed_test_click_health_monitors_category_20260522_164137.png



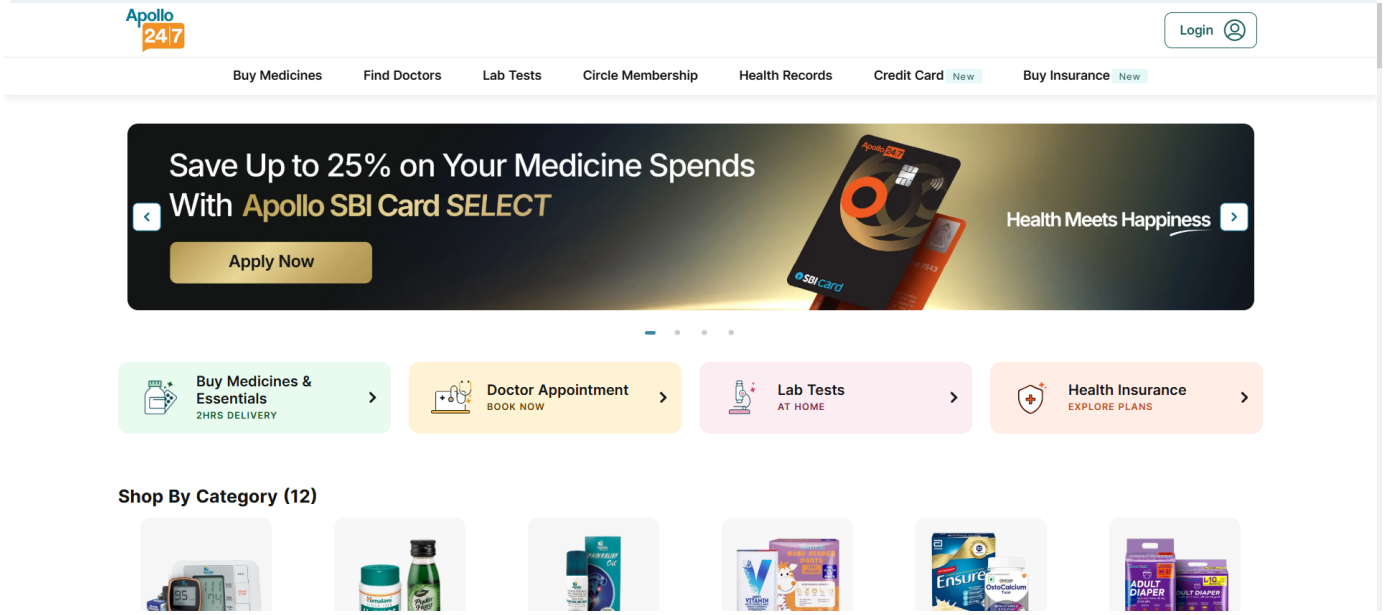
ApolloAutomation/reports/screenshots/passed_test_login_data0__20260522_164103.png



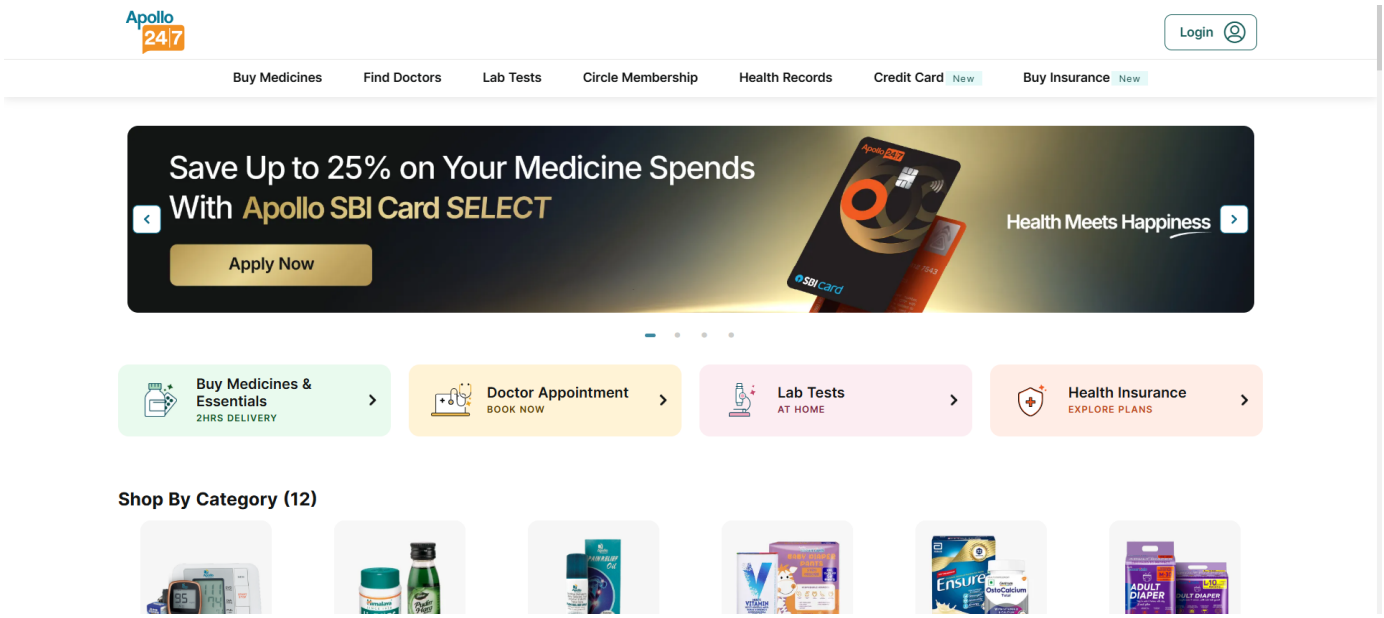
ApolloAutomation/reports/screenshots/passed_test_proceed_after_adding_product_to_cart_20260522_164409.png



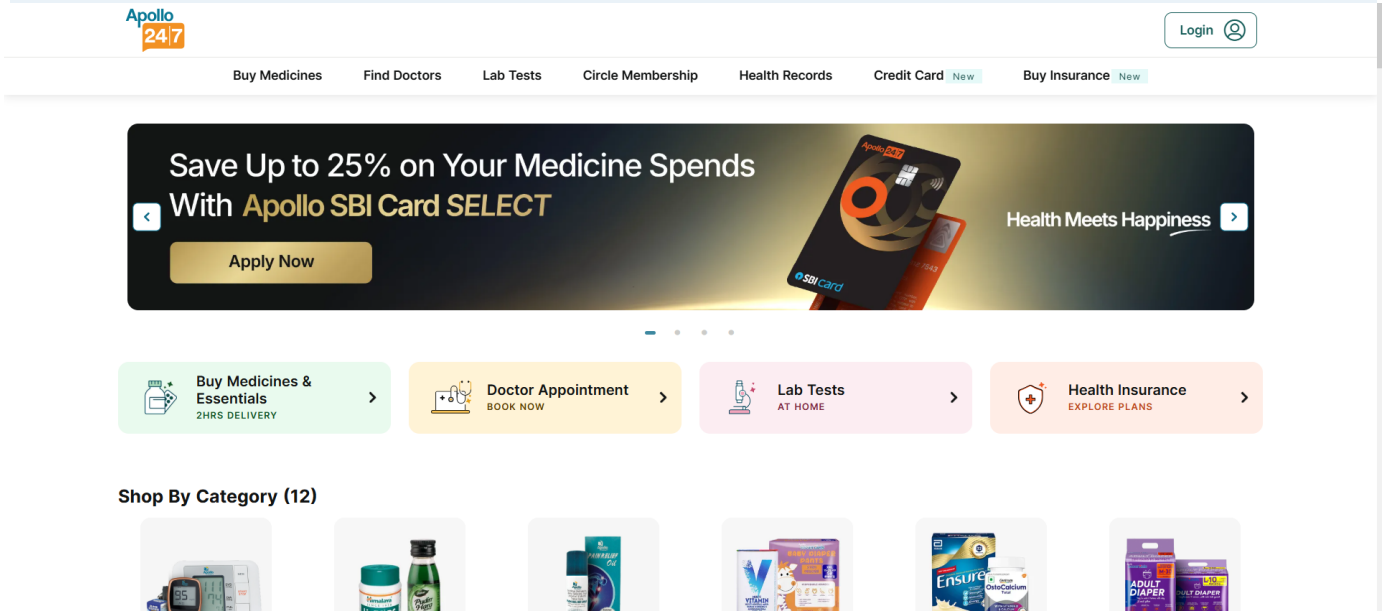
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data0__20260522_164106.png



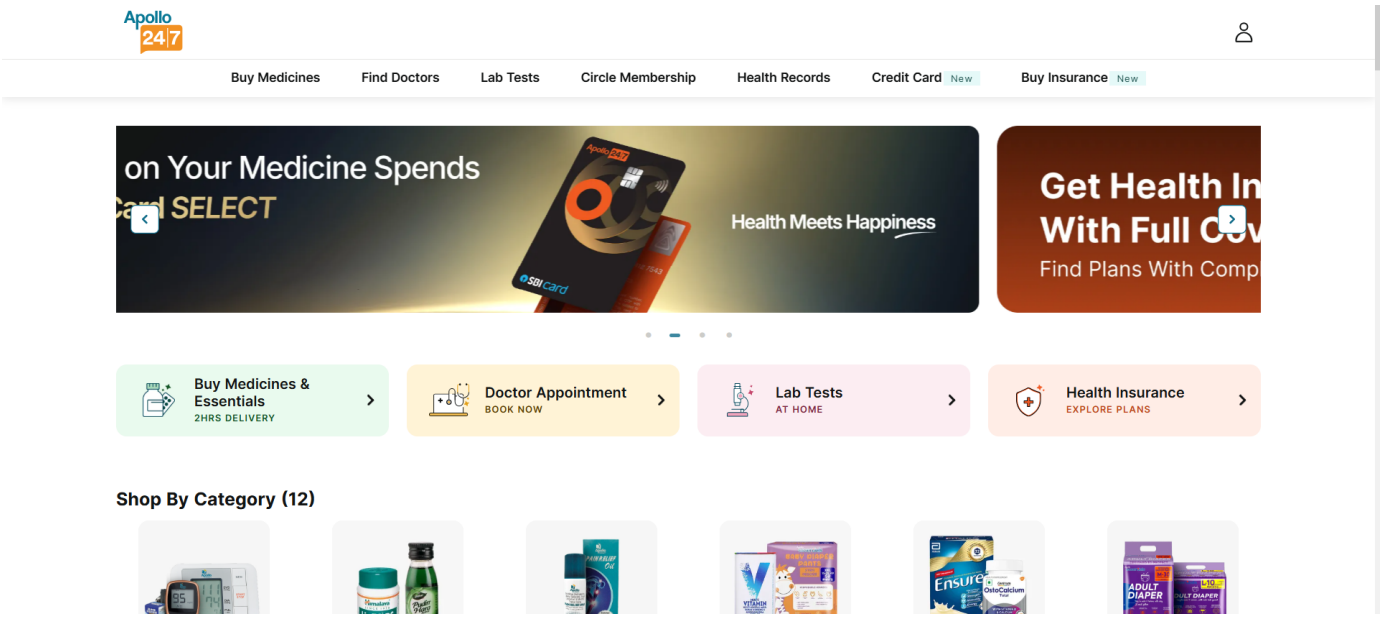
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data1__20260522_164108.png



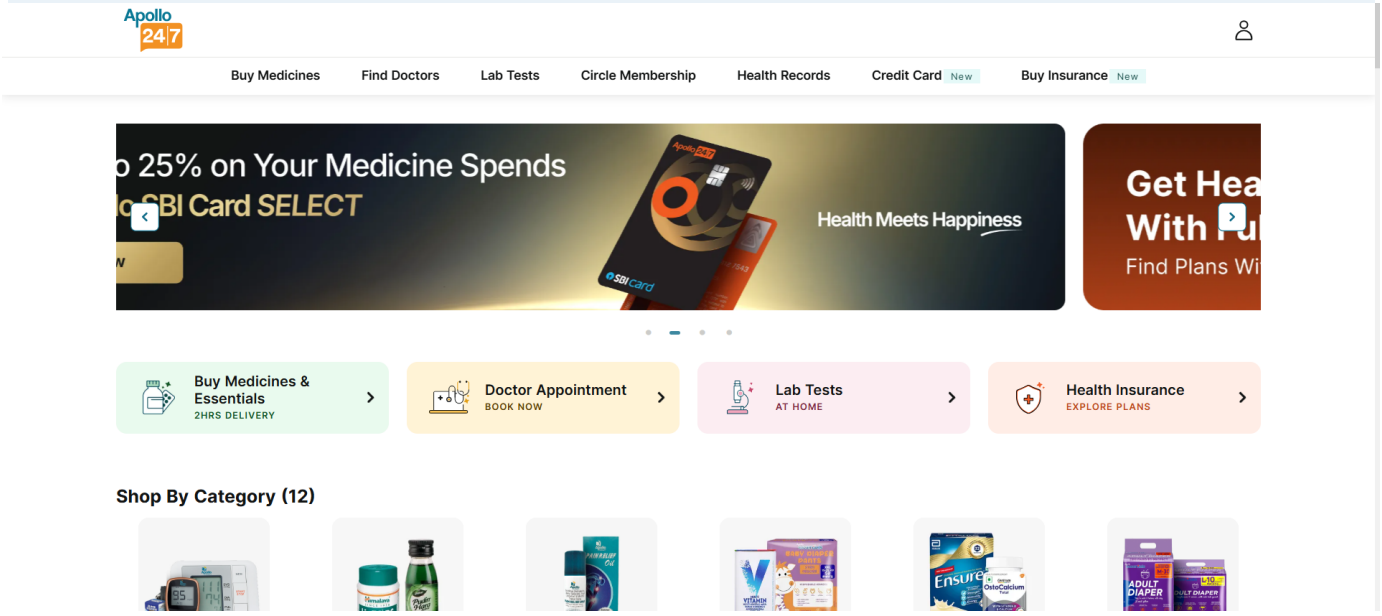
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data2__20260522_164109.png



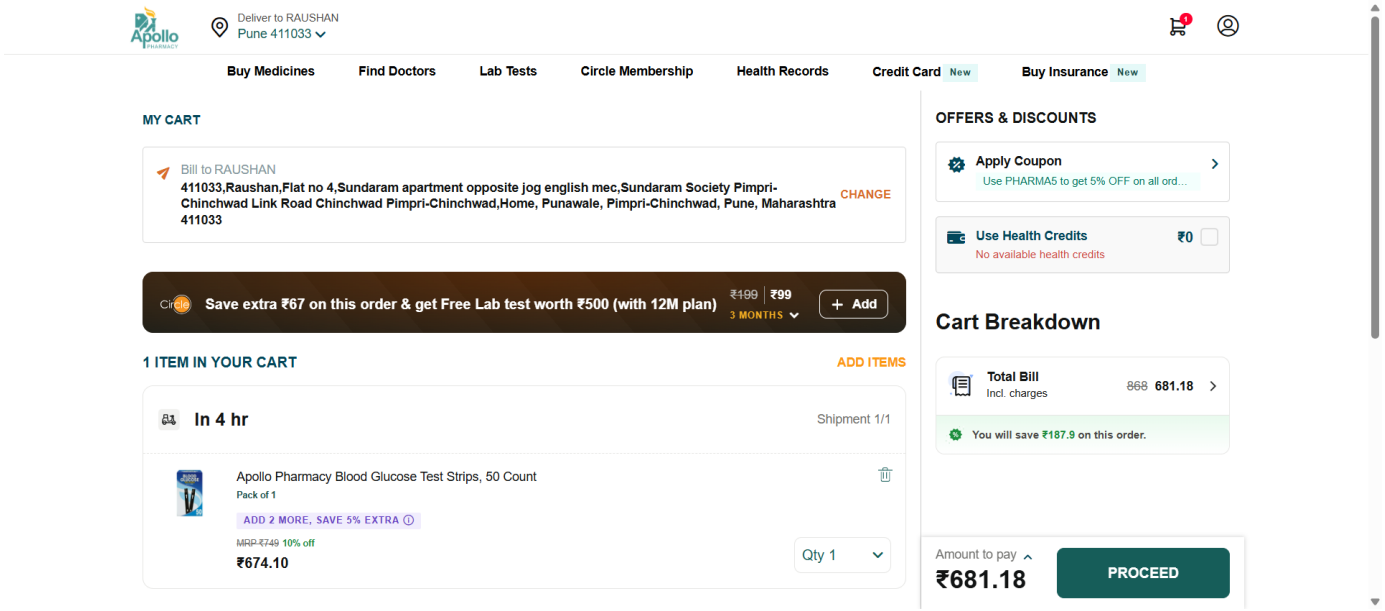
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data3__20260522_164114.png



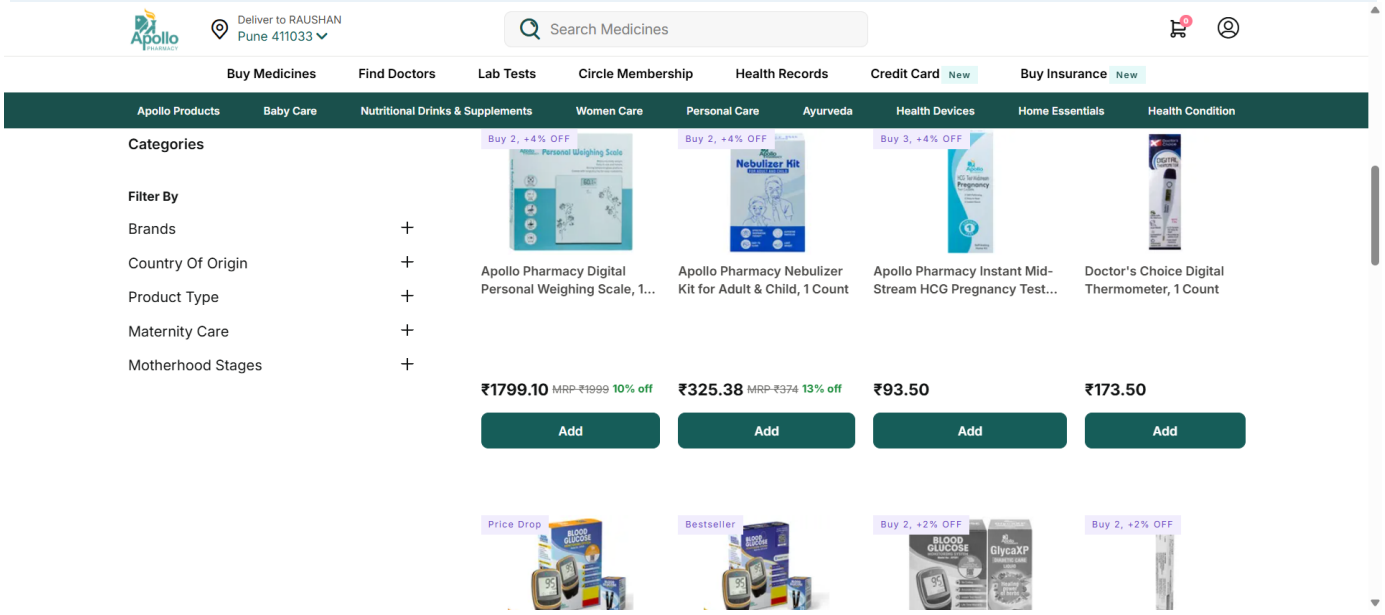
ApolloAutomation/reports/screenshots/passed_test_shop_by_category_data_data4__20260522_164118.png



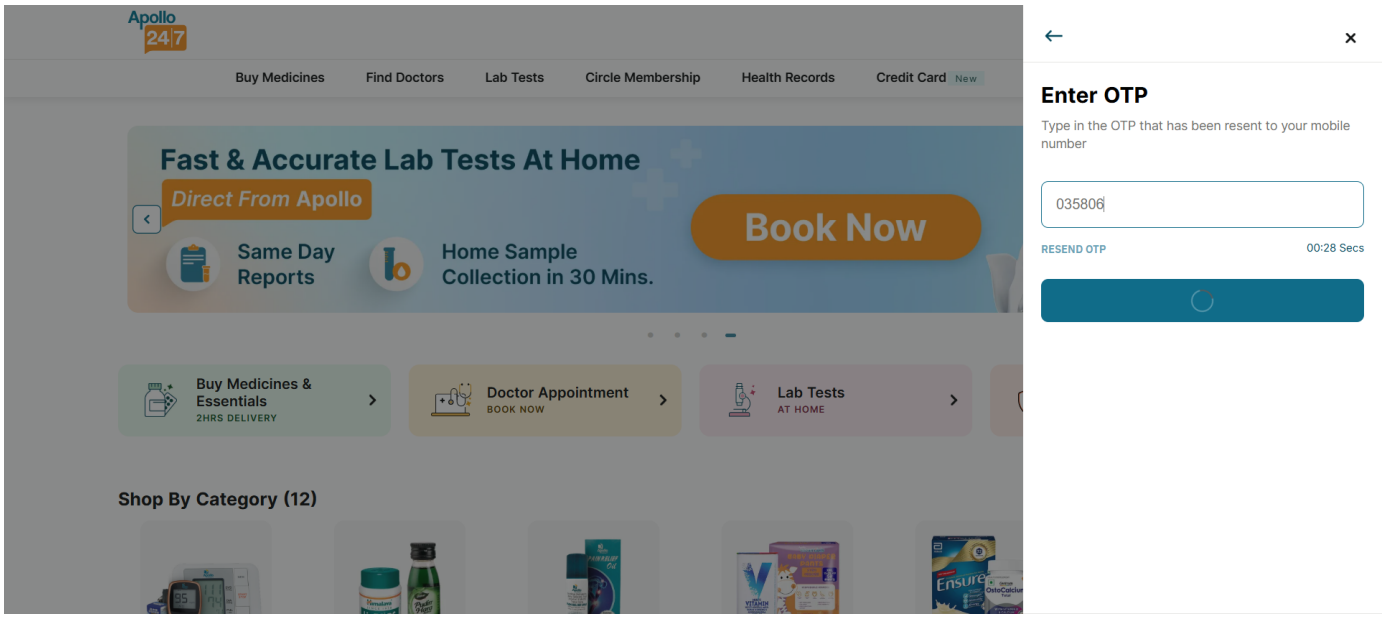
BBD_Apollo/reports/screenshots/passed_add_doctor_s_choice_product_to_cart_20260523_121334.png



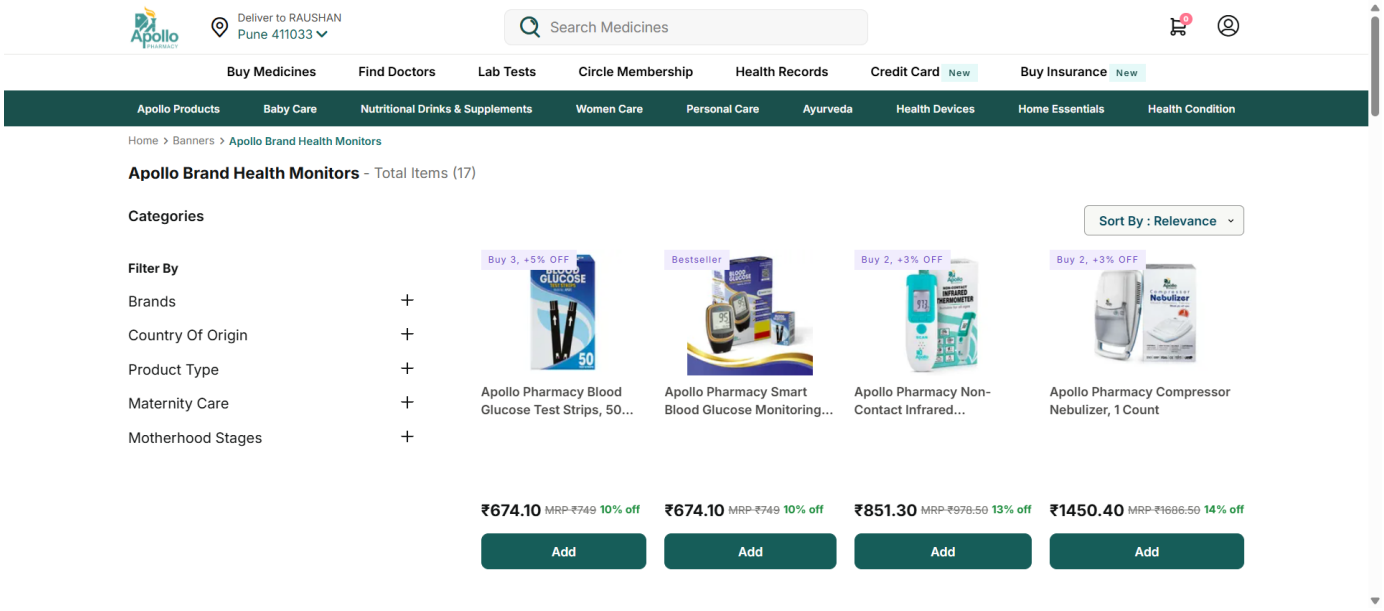
BBD_Apollo/reports/screenshots/passed_apply_doctor_s_choice_filter_20260523_121248.png



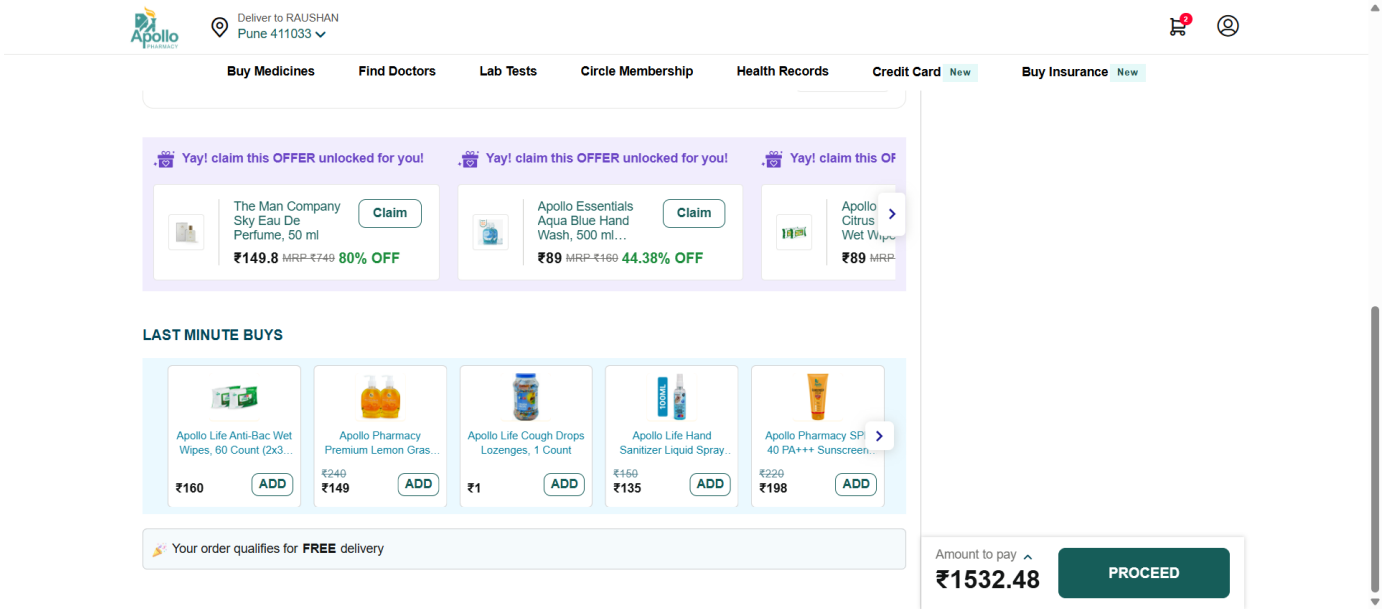
BBD_Apollo/reports/screenshots/passed_login_with_valid_mobile_number_and_otp_20260523_121142.png



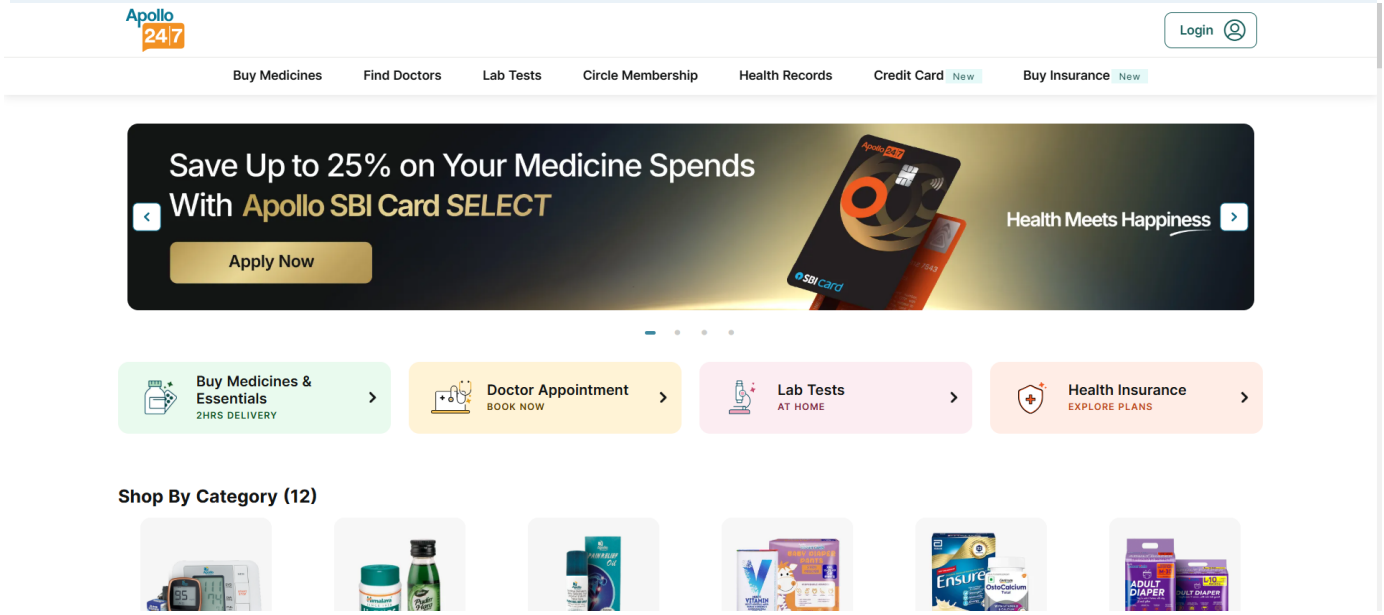
BBD_Apollo/reports/screenshots/passed_open_health_monitors_category_20260523_121214.png



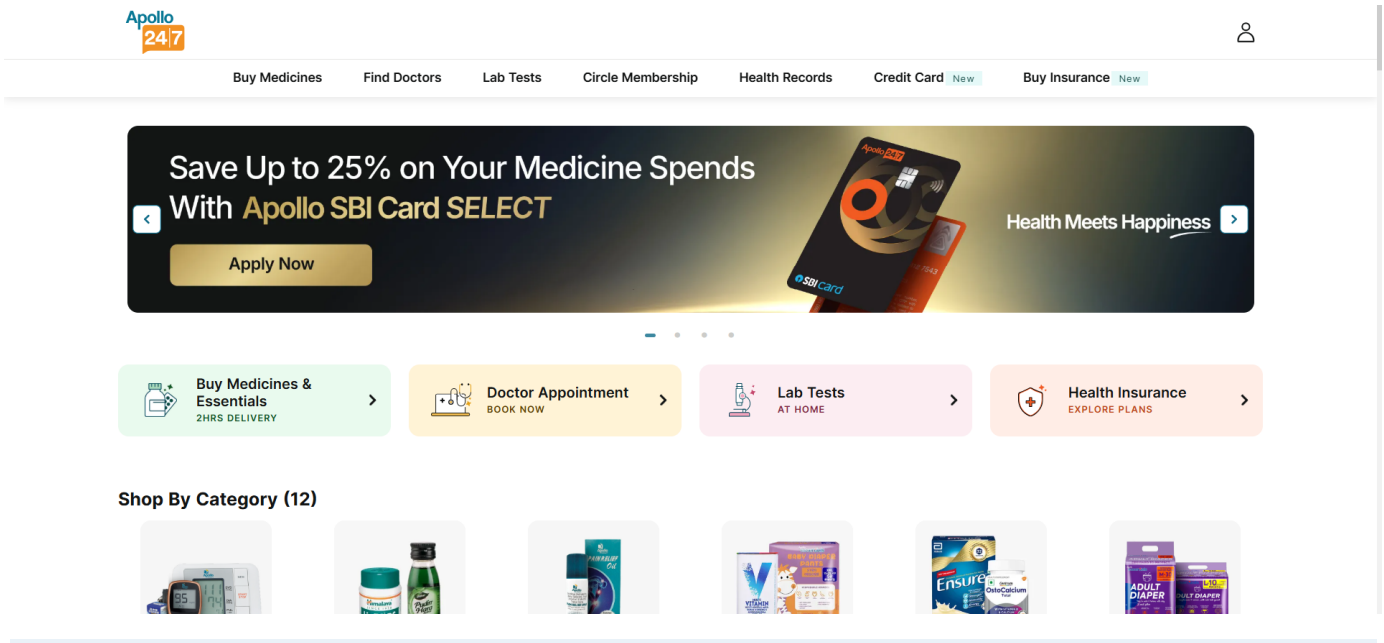
BBD_Apollo/reports/screenshots/passed_proceed_after_adding_product_to_cart_20260523_121433.png



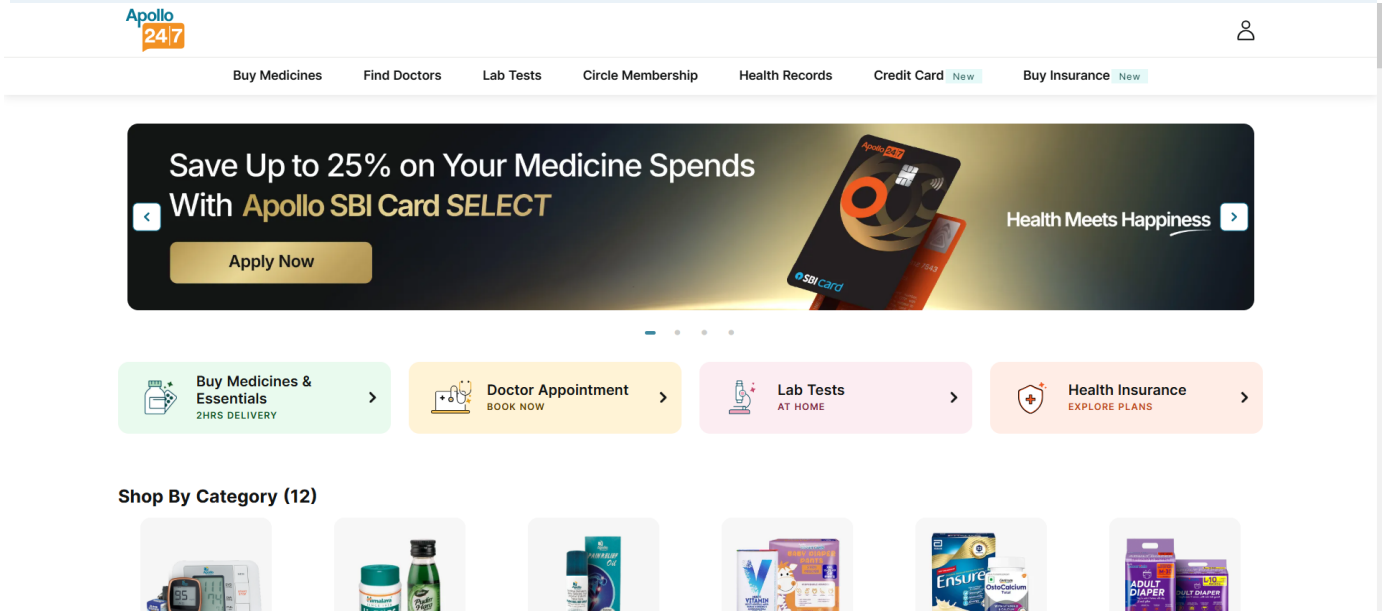
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.1__20260523_121144.png



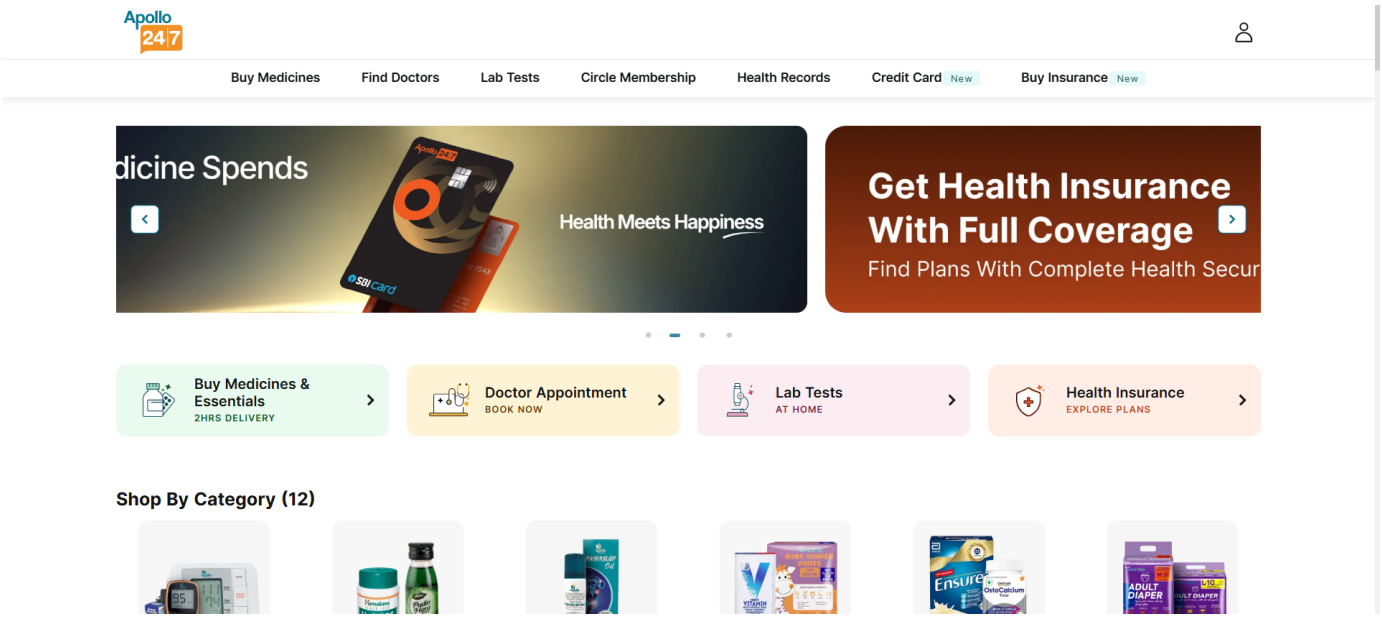
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.2__20260523_121146.png



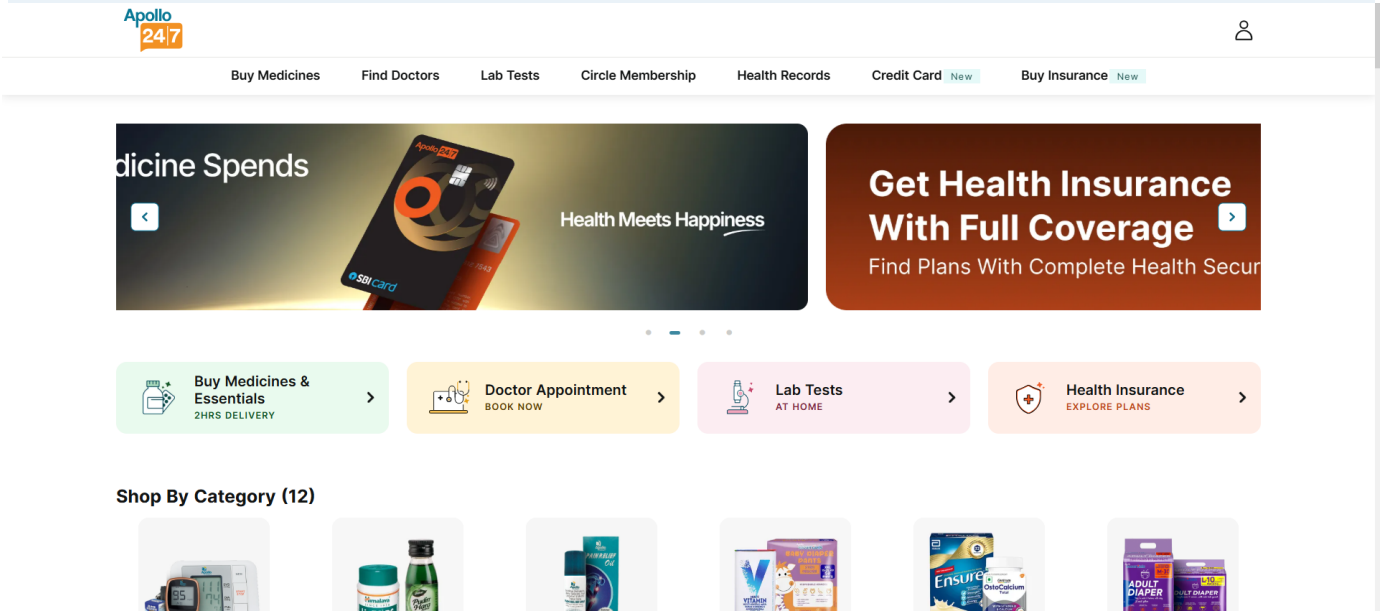
BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.3__20260523_121148.png



BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.4__20260523_121152.png



BBD_Apollo/reports/screenshots/passed_validate_category_visibility_--_@1.5__20260523_121156.png



6. Notes About This ZIP-to-PDF Conversion

- This PDF is a readable conversion of the uploaded ZIP, not a byte-for-byte archive replacement.

- Python virtual environments, Git internals, cache folders, and large binary web assets were excluded to keep the PDF useful and readable.

- All core project code, configuration, feature files, test data, logs, Allure summaries, and execution screenshots are included.

- Use the original ZIP file for exact runnable project restoration. Use this PDF for review, submission, documentation, and presentation.